

A day without new knowledge is a lost day.

- nologin.in/1
- https://codeshare.io/saleel

Database Technologies - MySQL

In this module we are going to learn SQL, PL/SQL and NoSQL(MongoDB)

- `sudo apt install build-essential`

MySQL is case-insensitive

SQL keywords → Case-insensitive

Table names → Depends on operating system (Maximum **64 characters** long)

Database/schema names → can also be OS-dependent (Maximum **64 characters** long)

Column names → Case-insensitive (Maximum **64 characters** long)

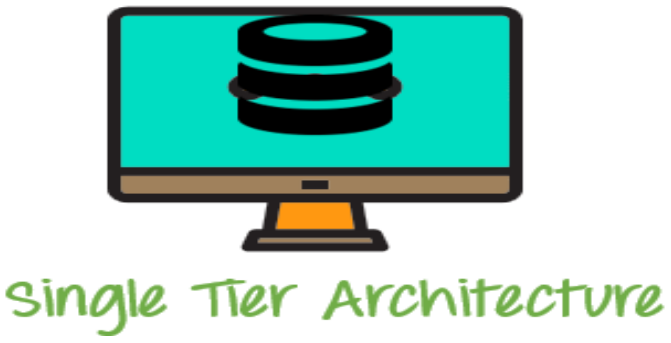
String values → Case-insensitive / Depends on collation (Collation defines how text (characters) are compared and sorted in a database.)

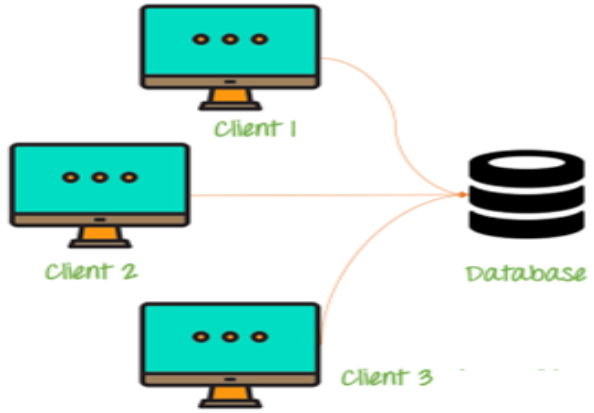
If A and a, B and b, ... are treated in the same way then it is case-insensitive.
MySQL is case-insensitive

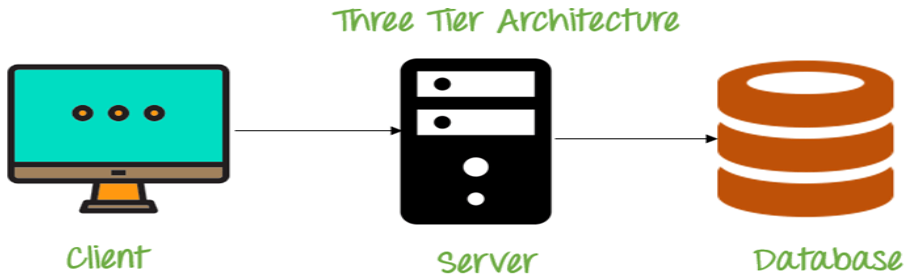
Introduction

If anyone who wants to develop a good application
then he should have the knowledge three major components.
They are . . . . .

- Presentation Layer [**UI**] [Is the part of an application through which the user interacts with the system.]
- Application Layer [**Server Application** and **Client Application**]
    1. A Server Application is a software application that **runs on a server** and **provides services** to other applications.
    2. A Client Application is a software application that **runs** on the **user's device** and allows the user to **interact** with a **server** or **service**.
- Data Layer [Data Access Object (DAO) / Data Access Layer (DAL)] { Flat Files | RDBMS | NoSQL }







Introduction

Why do we need databases (Use Case)?

We **need databases** because they **organize data** in a manner which allows us to store, query, sort, and manipulate data in various ways. Databases allow us to do all this things.

Many companies collects data from different resource (like Weather data, Geographical data, Finance data, Scientific data, Transport data, Cultural data (the ideas, customs, and social behaviour of a particular people or society), etc.)

- **A foreign key constraint is also known as a referential constraint or referential integrity constraint.**

What is Relation and Relationship?

Reference / Referential key

Remember:

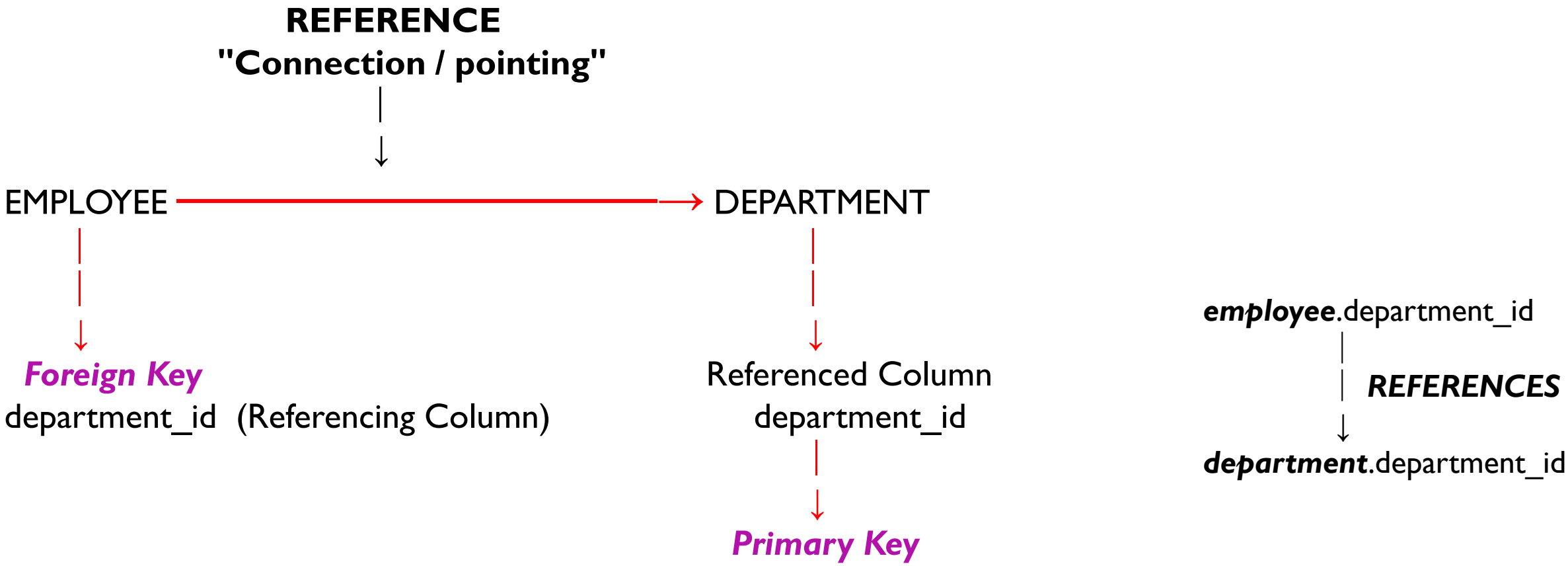
- A **reference** is a **relationship between** two tables where the **values** in **one table refer** to the **values in another table**. This is usually enforced **using a foreign key** constraint to maintain **referential integrity.**
  a) **The referencing column is called the Foreign Key of the child table.**
  b) **The referenced column is usually the Primary Key of the parent table.**

- A **referential key** is a column or set of columns in a table **that refers** to the **primary key** of another table. It establishes a relationship between two tables, where one table is called the **parent table**, and the other is called the **child table.**

reference / referential key

To understand the difference is to think as

1. ***"reference"*** as the relationship and
2. ***"referential key"*** as the key used to create that relationship.



relation and relationship?

Relation ***(in Relational Algebra "R" stands for relation)*****:** In Database,

- a relation represents a **table** or an **entity** than **contain attributes**.
- In Relational Algebra, a ***relation*** is a ***table*** with ***rows*** ***(***that ***share*** the ***same structure)*** and ***columns***,

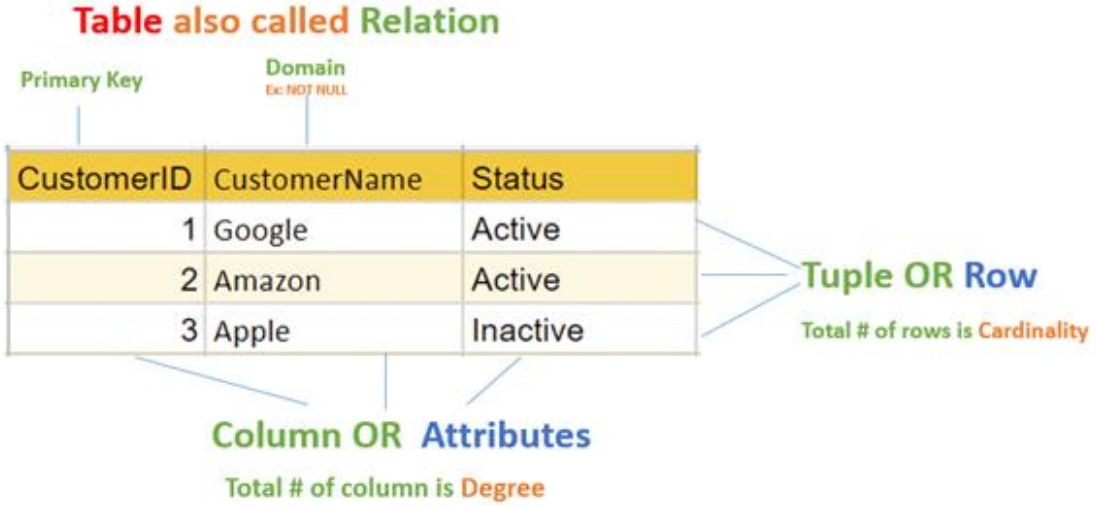
Relationship: In database, **relationship** is that how the **two entities** are **connected** to each other, i.e. what kind of ***relationship type*** they hold between them.

Primary/Foreign key is used to specify this relationship.

Remember:

Foreign Key is also known as

- referential constraint
- referential integrity constraint.



Note:

- **Table -** The table that physically stores the data of a relation.
- **Relation -** A logical construct that organizes data into rows and columns.

File Systems is the traditional way to keep your data organized.

File System vs DBMS

A File-Oriented System is the traditional way of storing and managing data before databases (like MySQL, Oracle, . . .) were introduced.

- Data is **stored** in **separate files** (text files, spreadsheets, binary files).
- Each application **manages** its own **files independently.**
- There is **no central control** over data.

file-oriented system

File Anomalies

```
struct Employee {
   int emp_no;
   char emp_name[50];
   int salary;
} emp[1000];
```

```
struct Employee {
   int emp_no;
   char emp_name[50];
   int salary;
};
struct Employee emp[1000];
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.       |
.       |
.       |
500 sam 3500
.       |
.       |
.       |
1000 amit 2300
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.       |
.       |
.       |
500 sam 3500
.       |
.       |
1000 amit 2300
.       |
.       |
2000 jerry 4500
.       |
.       |
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.       |
.       |
500  sam 3500
.       |
3   rajan 4500
.       |
500  sam 3500
.       |
.       |
1000 amit 2300
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.       |
.       |
sam 500 3500
.       |
ram  550 5000
.       |
1000 amit 2300
```

c:\employee.txt

```
1   suraj 4000
2   ramesh 6000
3   rajan 4500
.       |
500 sam 3500
.       |
600 neel 4500
```

- Create/Open an existing file
- Reading from file
- Writing to a file
- Closing a file

file-oriented system

File Anomalies

c:\employee.txt

1 suraj 4000
2 ramesh 6000
3 rajan 4500
.
.
.
500 sam 3500
.
.
.
1000 amit 2300

file attributes

- File Name
- Type
- Location

file permissions

- File permissions
- Share permissions

search empl ID=1

1 suraj 4000
2 ramesh 6000
3 rajan 4500
.
.
.
500 sam 3500
.
.
.
1000 amit 2300

search emp_name

1 suraj 4000
2 ramesh 6000
3 rajan 4500
.
.
.
500 sam 3500
.
.
.
1000 amit 2300

advantages & disadvantage of file-oriented system

advantages of file-oriented system

The biggest advantage of file-based storage is as follows.

- **Backup**: It is possible to take faster and automatic back-up of database stored in files of computer-based systems.
- **Data retrieval:** It is possible to retrieve data stored in files in easy and efficient way.
- **Flexibility:** File systems provide flexibility in storing various types of data, including text **documents, images, audio, video,** and more
- **Cost-Effectiveness**: File systems often do not incur **licensing costs**, making them cost-effective for basic data storage needs.
- **Editing**: It is easy to edit any information stored in computers in form of files.
- **Remote access**: It is possible to **access data** from **remote location**.
- **Sharing**: The files stored in systems can be **shared** among **multiple users** at a same time.

disadvantage of file-oriented system

The biggest disadvantage of file-based storage is as follows.

- **Data redundancy**: It is possible that the same **information** may be **duplicated** in different files. This leads to data redundancy results in memory wastage.
  (Suppose a customer having both kind of accounts - **saving** and **current** account. In such a situation a customer detail are stored in both the file, saving.txt- file and current.txt- file , which leads to Data Redundancy.)

- **Data inconsistency**: Because of data redundancy, it is possible that data may **not** be in **consistent state**.
  (Suppose customer changed his/her address. There might be a possibility that address is changed in only one file (saving.txt) and other (current.txt) remain unchanged.)

- **Data Integrity**: Data integrity **refers** to the **accuracy** and **consistency** of **data**. In a file-oriented system, enforcing data integrity is **difficult** because there are **no built-in** mechanisms to ensure that data is valid or consistent across multiple files.
  (the balance field value must be grater than 5000.)

- **Concurrency Issues:** When **multiple users** or applications **try** to **access** and **modify** **a file at the same time**, concurrency problems can arise.
  (if two users attempt to update the same file simultaneously, it can lead to data corruption or loss of data.)

disadvantage of file-oriented system

- **Limited data sharing**: Data are scattered in various files and also different files may have different formats (for example: .txt, .csv, .tsv and .xml) and these files may be **stored** in **different folders** so, **due** to this it is **difficult** to **share** data **among different** applications also if the **saving account department** wants to share data with **loan department,** they need to manually copy files, leading to delays because File Systems do not support multi-user environments.

- **Data Isolation:** Because data are scattered in various files, and files may be in different formats (for example: .txt, .csv, .tsv and .xml), writing new application programs to retrieve the appropriate data is difficult.
  (Suppose a loan data is in one file and account holder data in another, there is no easy way to **analyze** account holder data with his loan status.)

- **Data security:** Data should be secured from **unauthorized access**, for example a account holder in a bank should not be able to see the account details of another account holder, such kind of security constraints are difficult to apply in file processing systems.

Relation Schema: A relation schema represents **name** of the **relation** with its **attributes**, every attribute would have an associated **domain**.

e.g.

- ***Student***(rollNo:INT, name:VARCHAR(20), address:VARCHAR(50), phone:VARCHAR(12), age:INT, PRIMARY KEY(rollNo)) is relation schema for STUDENT
- ***Customers***(CustomerID:INT, Name:VARCHAR(50), Email:VARCHAR(100), City:VARCHAR(50), PRIMARY KEY(CustomerID)) is relation schema for CUSTOMERS

DBMS

- **database:** Is the **collection** of **related data** which is **organized**, database **can store** and **retrieve** large amount of data easily, which is stored in **one** or **more data files** by **one** or **more users**, it is called as **structured data.**
- **management system**: it is a software, designed to **define**, **manipulate**, **retrieve** and **manage data** in a database.

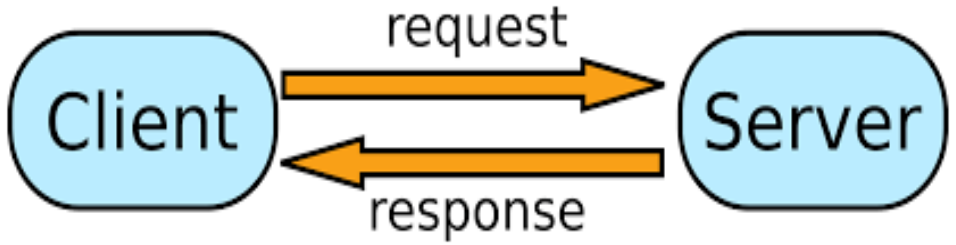


relational database management system?

A RDBMS is a database management system (DBMS) that is based on the **relational model** introduced by Edgar Frank Codd at IBM in 1970.

RDBMS supports

- *client/server Technology*
- *Highly Secured* [***Database security ensures that only authorized users can access the data***]
- *Relationship (PK/FK)*







object relational database management system?

An object database is a database management system in which information is represented in the form of objects.

PostgreSQL is the most popular pure ORDBMS. Some popular databases including Microsoft SQL Server, Oracle, and IBM DB2 also support objects and can be considered as ORDBMS.

Advantage of ORDBMS

- Function/Procedure **overloading**.
- Extending server functionality with external functions written in **C** or **Java**.
- **User defined data types**.
- **Inheritance** of **tables** under **other** tables.

object relational database management system?

- CREATE or REPLACE TYPE ***address*** AS OBJECT(city VARCHAR2(10), state VARCHAR2(2));
- CREATE TABLE person(id INT, name VARCHAR2(10), addr ***address***);
- INSERT INTO person VALUES(1, 'saleel', address('baroda', 'GJ'));
- SELECT id, name, ***n***.addr.city FROM person ***n***;

- CREATE or REPLACE TYPE ***city*** AS VARRAY(3) OF VARCHAR2(10);
- CREATE TABLE x (id INT, ename VARCHAR2(10), c ***city***);
- INSERT INTO x values(1, 'saleel', ***city***('baroda', 'surat', 'bharuch'));
- SELECT n.id, n.ename, nn.column_value FROM x n, TABLE(n.c) nn;
- SELECT id, ename, nn.column_value FROM a n, TABLE(n.c) nn WHERE nn.column_value = 'blue';

relational model concepts
and
properties of relational table

relational model concepts

- **Relation (Table)** – In relational model, relations are saved in the form of Tables. A table has rows and columns.
- **Attribute (Column)** – Attributes are the properties that define a relation. **e.g.** (**roll_no, name, address, age, . . .)**
- **Tuple (Row/Record)** – A single row of a table, which contains a single record for that relation is called a tuple.
- **Relation schema** – A relation schema describes the Relation Name (Table Name), Attributes (Column Names), Domain of Attributes (Data Types & Allowed values), Constraints (Primary Key, Foreign Key, etc.).

  e.g. **Customers**(CustomerID:INT, Name:VARCHAR(50), Email:VARCHAR(100), City:VARCHAR(50), PRIMARY KEY(CustomerID)) is relation schema for CUSTOMERS
- **Attribute domain** – An attribute domain in a relational database refers to the set of allowed values for an attribute (column). It defines the data type and constraints that restrict the values an attribute can take.

Remember:

- In database management systems, ***NULL*** (***absence of a value***) is used to represent **missing** or **unknown** data in a table column.

properties of relational table

ID (PK)	job	firstName	DoB	sal
1	manager	Saleel Bagde	yyyy-mm-dd	••••••
3	salesman	Sharmin	yyyy-mm-dd	••••••
4	accountant	Vrushali	yyyy-mm-dd	ABC
2	salesman	Ruhan	yyyy-mm-dd	••••••
5	9500	manager	yyyy-mm-dd	••••••
5	Salesman	Rahul Patil	yyyy-mm-dd	NULL

Properties of Relational Tables:

- Values are atomic (no multivalued cells).
- Column values are of the same kind. (***Attribute Domain*: Every attribute has some pre-defined datatypes, format, constraints of a column, and defines the range of values that are valid for that column known as attribute domain.)**
- Each row is unique.
- The sequence/order of columns is irrelevant – (unimportant).
- The sequence/order of rows is irrelevant – (unimportant).
- Each table name, attribute/column must have a unique name.
- Attributes may contain **NULL** (unknown/missing values)

What is data?

what is data?

Data is any raw facts that can be stored and that can be processed by a computer.

Data can be in the form of Text or Multimedia

e.g.

- number, characters, or symbol
- images, audio, video, or signal



What is Entity Relationship Diagram?

Entity Relationship Diagram (ER Diagram)

Use E-R model to get a **high-level** **graphical view** to **describe** the **"ENTITIES"** and their **"RELATIONSHIP"**

The basic constructs/**components** of ER Model are **Entity**, **Attributes** and **Relationships**.

An entity can be a **real-world object.**

What is Entity?

An entity in DBMS is a real-world object that has certain properties called attributes that define the nature of the entity.

entity

In relation to a database , an entity is a

- Person(student, teacher, employee, client, department, …)
- Place(classroom, building, …) --a particular position or area
- Thing(computer, lab equipment, …) --an object that is not named (represents a tangible object)
- Concept(course, batch, student's attendance, …) -- an idea,

about which data can be stored. All these entities have some **attributes** or **properties** that give them their **identity**.

Every entity has its own characteristics.

When you are designing attributes for your entities,

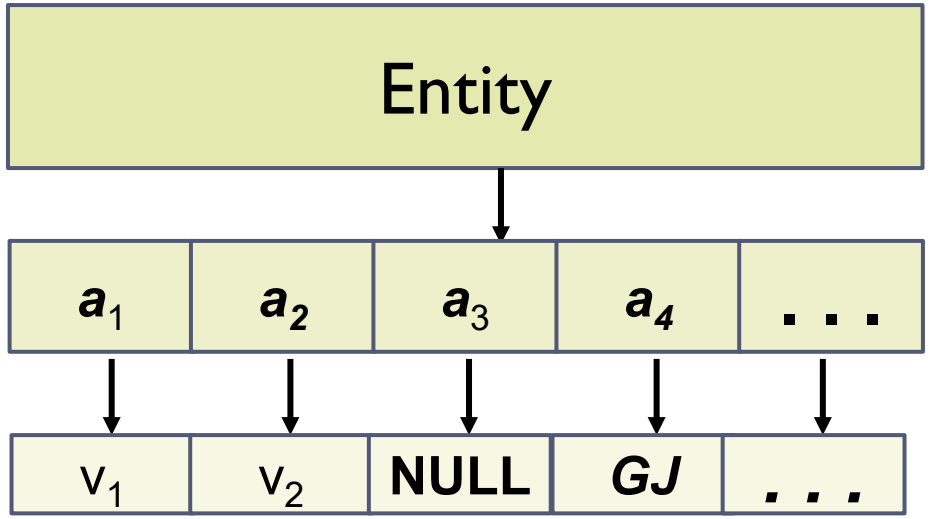
- **you will sometimes find that an attribute does not have a value.**
- For example, you might want an attribute for a person's middle name, but you can't require a value because some people have no middle name. **For these, you can define the attribute so that it can contain *NULL* (*absence of a value*) values.**

What is an Attribute?

Attributes are the properties that define a relation.

e.g. **Student**(***rollNo***:INT, ***name***:VARCHAR(20), ***address***:VARCHAR(50), ***age***:INT)

In some cases, you might not want a specific attribute to contain a NULL value, but you don't want to require that the user or program always provide a value. In this case, a default value might be appropriate. **A default value is a value that applies to an attribute if no other valid value is available.**



In Entity Relationship(ER) Model attributes can be classified into the following types.

- Simple/Atomic and Composite Attribute
- Single Valued and Multi Valued attribute
- Stored and Derived Attributes
- Complex Attribute

Remember:

In SQL, the same name can be used for two (or more) attributes as long as the attributes are in different relations.

attributes

- **Simple / Atomic Attribute** --VS-- **Composite Attribute**
  (Can't be divided further) (Can be divided further)
- **Single Value Attribute** --VS-- **Multi Valued Attribute**
  (Only One value) (Multiple values)
- **Stored Attribute** --VS-- **Derived Attribute**
  (Only One value) (Virtual)
- **Complex Attribute**
  (Composite & Multivalued)

Employee ID: An employee ID can be a composite attribute, which is composed of sub-attributes such as department code, job code, and employee number.

- **Atomic Attribute:** An attribute that cannot be divided into smaller independent attribute is known as atomic attribute.
  e.g. ID's, PRN, age, gender, zip, marital status cannot further divide.
- **Single Value Attribute:** An attribute that holds exactly one value for a given record at any point in time is known as single valued attribute. Single-valued attributes are typically used to provide a unique identifier for a record.
  e.g. manufactured part can have only one serial number, voter card ID, blood group, branchID can have only one value.
- **Stored Attribute:** The stored attribute are such attributes which are already stored in the database and from which the value of another attribute is derived.
  e.g. (HRA, DA…) can be derive from salary, age can be derived from DoB, total marks or average marks of a student can be derived from marks, TotalPrice can be derived from Quantity × UnitPrice.

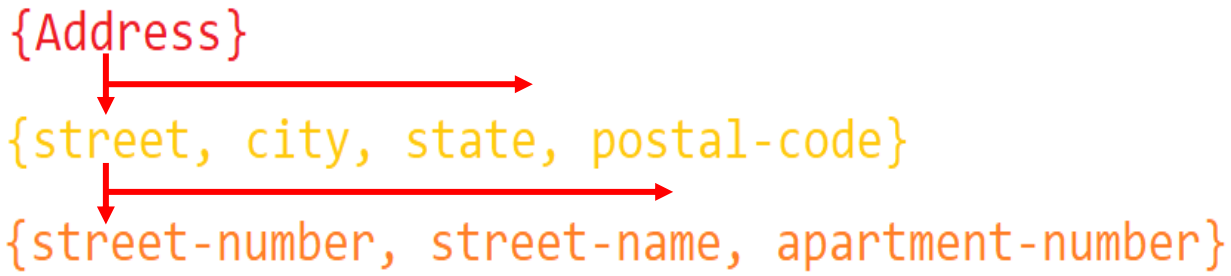
Composite **VS** Multi Valued Attribute

Composite Attribute

composite / multi valued attributes

Person Entity

- *Name* attribute: (`firstName` + `middleName` + `lastName`)
- *PhoneNumber* attribute: (`countryCode` + `cityCode` + `phoneNumber`)
- *Date* attribute: (`Day` + `Month` + `Year`)
- *productDimensions* attribute: (`Length` + `Width` + `Height`)
- *carRegistration* attribute: (`State_Code` + `Series` + `Number`)



Multi Valued Attribute

Person Entity

- *Hobbies* attribute: [reading, hiking, hockey, skiing, photography, . . .]
- *SpokenLanguages* attribute: [Hindi, Marathi, Gujarati, English, . . .]
- *Degrees* attribute**:** [10th, 12th, BE, ME, PhD, . . .]
- *emailID* attribute**:** [saleel@gmail.com, salil@yahoomail.com, . . .]
- *Skills* attribute: [MySQL, Oracle, Redis, MongoDB, Java, . . .]

types of Keys?

Keys are used to establish relationships between tables and also to uniquely identify any record in the table.

types of Keys?

r = Employee(EmployeeID, FullName, job, salary, PAN, DateOfBirth, emailID, deptno)

- **Candidate Key:** are individual columns in a table that qualifies for uniqueness of all the rows. Here in Employee table EmployeeID, PAN or emailID are Candidate keys.
- **Primary Key**: is the columns you choose to maintain uniqueness in a table. Here in Employee table you can choose either EmployeeID, PAN or emailID columns, EmployeeID is preferable choice.
- **Alternate Key**: Candidate column other the primary key column, like if EmployeeID is primary key then , PAN or emailID columns would be the Alternate key.
- **Super Key**: If you add any other column to a primary key then it become a super key, like EmployeeID + FullName or EmployeeID + deptno is a Super Key.
- **Composite Key**: If a table do not have any single column that qualifies for a Candidate key, then you have to select 2 or more columns to make a row unique. Like if there is no EmployeeID, PAN or emailID columns, then you can make FullName + DateOfBirth as Composite key. But still there can be a narrow chance of duplicate row. Ensures data uniqueness in many-to-many relationships. e.g. in order_details table we can have multiple products OrderID + ProductID

What is a Prime, Non-Prime Attribute?

Prime attribute (*Entity integrity*):- An attribute, which is a **part of the prime-key** (candidate key), is known as a prime attribute.

Consider a relation Student(StudentID, Name, Email, Phone).

- ***Candidate Keys:*** {StudentID}, {Email}, {Phone}
- ***Prime Attributes:*** StudentID, Email, Phone (since they are part of a Candidate Key).

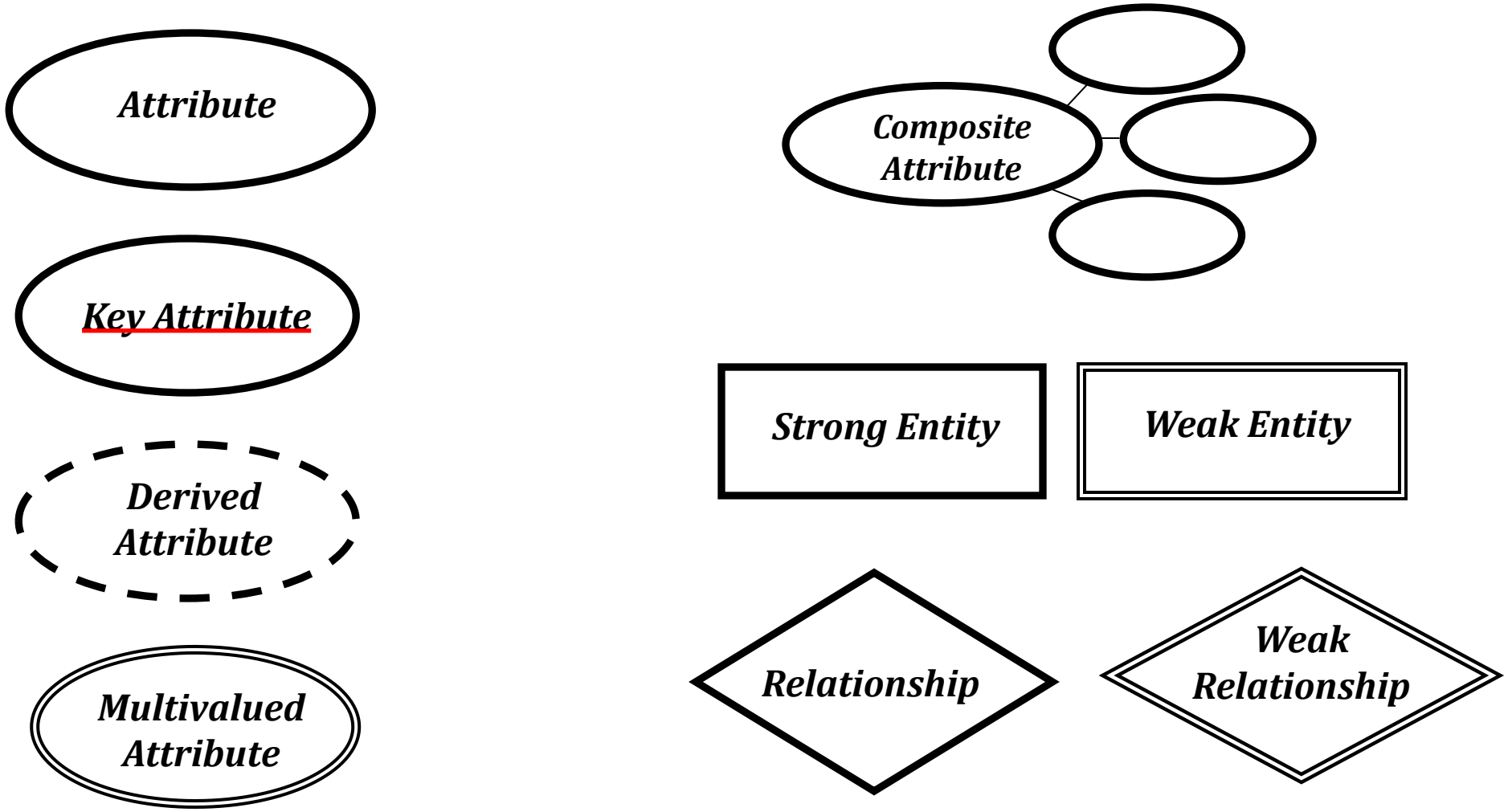
Non-prime attribute:- An attribute, which is **not a part of the prime-key** (candidate key), is said to be a non-prime attribute.

In the Student(StudentID, Name, Email, Phone) relation:

- ***Candidate Keys:*** {StudentID}, {Email}, {Phone}
- ***Prime Attributes:*** StudentID, Email, Phone
- ***Non-Prime Attribute:*** Name (because it is not part of any Candidate Key).

Entity Relationship Diagram Symbols

entity relationship diagram symbols



strong and weak entity

Strong Entity: A strong entity is not dependent on any other entity in the schema. A strong entity will always have a primary key. Strong entities are represented by a single rectangle.

Weak Entity: A weak entity is dependent on a strong entity to ensure its existence. Unlike a strong entity, a weak entity does not have any primary key. A weak entity is represented by a double rectangle. The relation between one strong and one weak entity is represented by a double diamond. This relationship is also known as ***identifying relationship***.

Example 1 – A loan entity can not be created for a customer if the customer doesn't exist

Example 2 – A payment entity can not be created for a loan if the loan doesn't exist

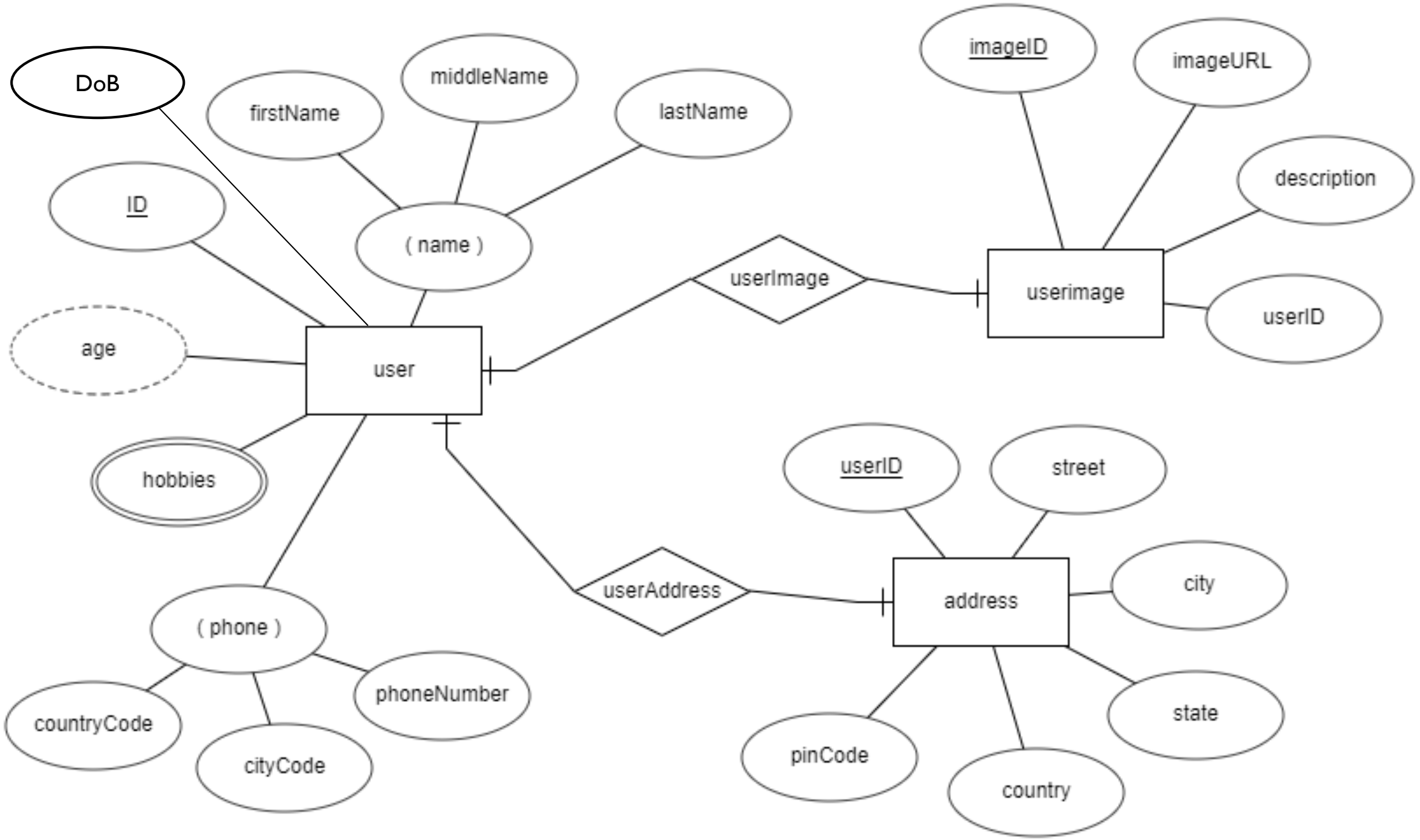
Example 3 – A customer address entity can not be created for the customer if the customer doesn't exist

Example 4 – A prescription entity can not be created for a patient if the patient doesn't exist

strong and weak entity

Strong Entity	Weak Entity
— Order (OrderID)	— OrderItem (ItemID, OrderID)
— University (UniID)	— Scholarship (ScholarshipID, UniID)
— Patient (PatientID)	— MedicalRecord (RecordID, PatientID)
— Account (AccountID)	— Transaction (TransactionID, AccountID)
— Student (StudentID)	— Grade (GradeID, StudentID)
— Vehicle (VehicleID)	— InsurancePolicy (PolicyID, VehicleID)
— Hotel (HotelID)	— RoomBooking (BookingID, HotelID)
— Product (ProductID)	— WarrantyClaim (ClaimID, ProductID)
— Student (StudentID)	— AttendanceRecord (RecordID, StudentID)
TODO	TODO
TODO	TODO
TODO	TODO
TODO	TODO

entity relationship diagram



What is a degree, cardinality and union in database?

What is a degree, cardinality and union in database?

r = Employee(EmployeeID, FullName, job, salary, PAN, DateOfBirth, emailID, deptno)

- **Degree d(R) / Arity**: Total number of **attributes/columns** present in a relation/table is called **degree of the relation** and is denoted by **d(R)**.
  1. **Degree / Arity =** ***Number of columns***
  2. **d(Employee) =** *8*

- **Cardinality |R|**: Total number of **tuples/rows** present in a relation/table, **is called cardinality of a relation** and is denoted by $|R|$, which changes dynamically as rows are inserted or deleted.
  1. **Cardinality |R| =** ***Number of rows/records in relation |R|.***

  Cardinality is the numerical relationship between rows of one table and rows in another. Common cardinalities include ***one-to-one, one-to-many,*** and ***many-to-many.***

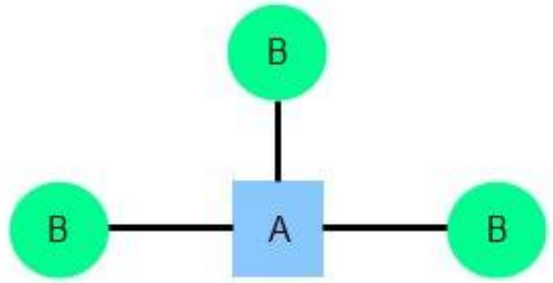
- **Union Compatibility**: Two relations ***R*** and ***S*** are set to be Union Compatible to each other if and only if:
  1. ***They have the same degree d(R).***
  2. ***Domains of the respective attributes should also be same.***
  3. ***Column names don't need to match, but positions must match.***

Common relationship

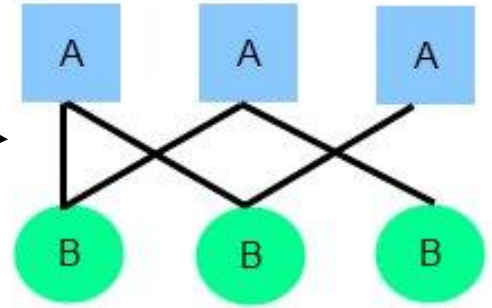
1. one-to-one (1:1)



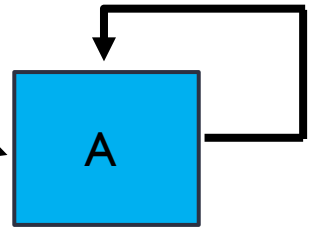
2. one-to-many (1:M)



3. many-to-many (M:N)



4. Self-Referencing (Recursive)



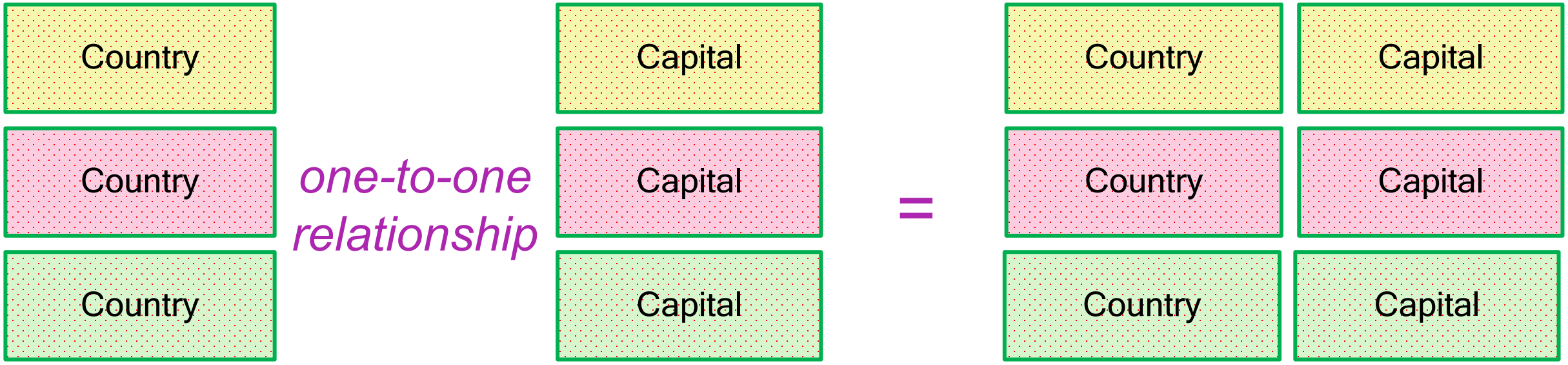
one-to-one relationship

One row in **Table A** corresponds to **one and only one** row in **Table B**.

one-to-one relationship

A *one-to-one* relationship between two tables means that a row in one table can only relate to zero/one row in the table on the other side of their relationship. This is the least common database relationship.

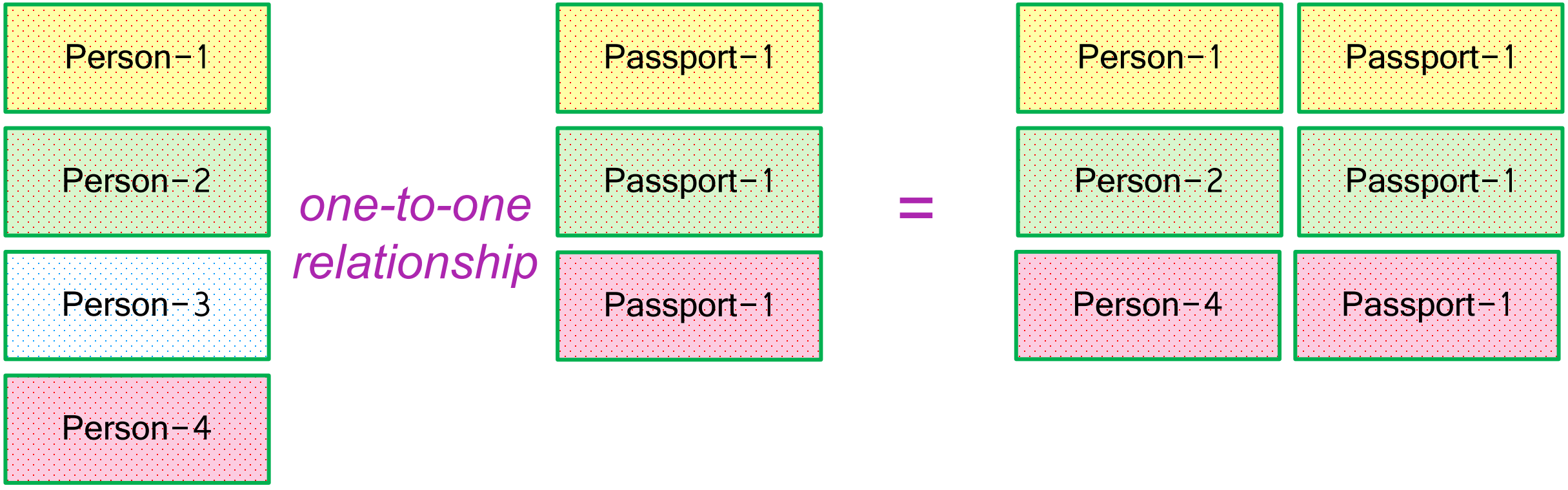
A *one-to-one* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which one element of entity ***R*** may only be linked to zero/one element of entity ***S***, and vice versa.



one-to-one relationship

A *one-to-one* relationship between two tables means that a row in one table can only relate to zero/one row in the table on the other side of their relationship. This is the least common database relationship.

A *one-to-one* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which one element of entity ***R*** may only be linked to zero/one element of entity ***S***, and vice versa.



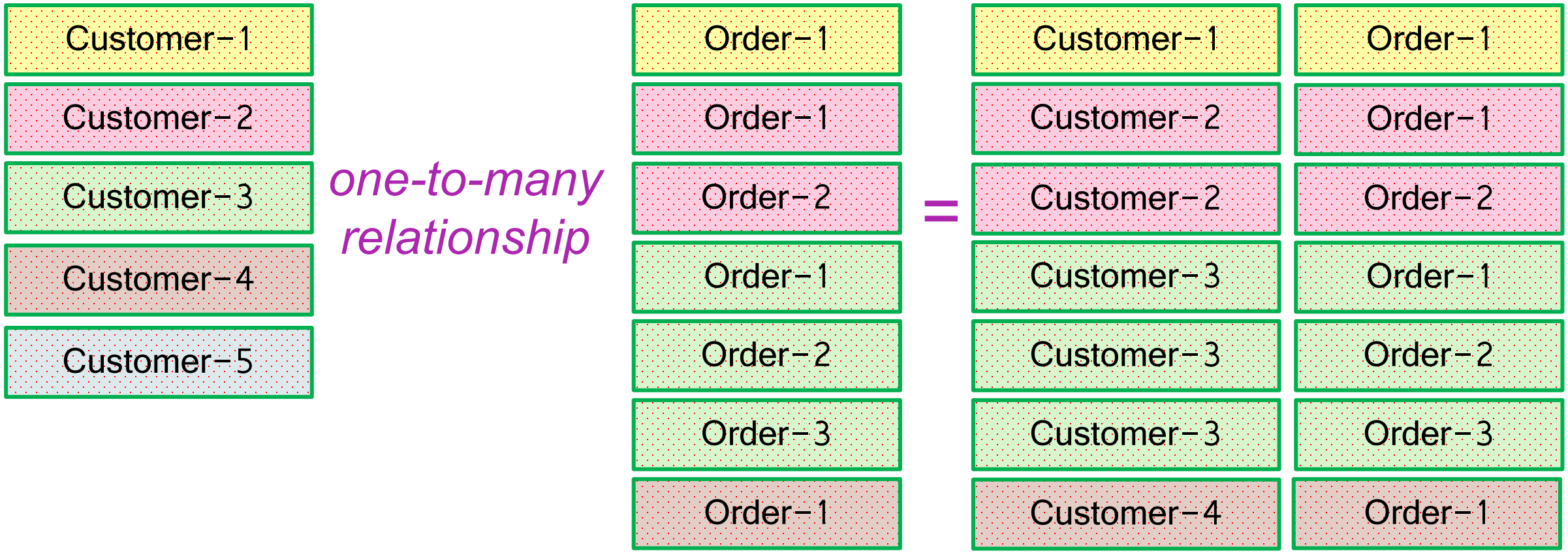
one-to-many relationship

One row in **Table A** can relate to **many rows in Table B**, but **many rows in Table** B **belongs to only one Table A**.

one-to-many relationship

A *one-to-many* relationship between two tables means that a row in one table can have zero or more row in the table on the other side of their relationship.

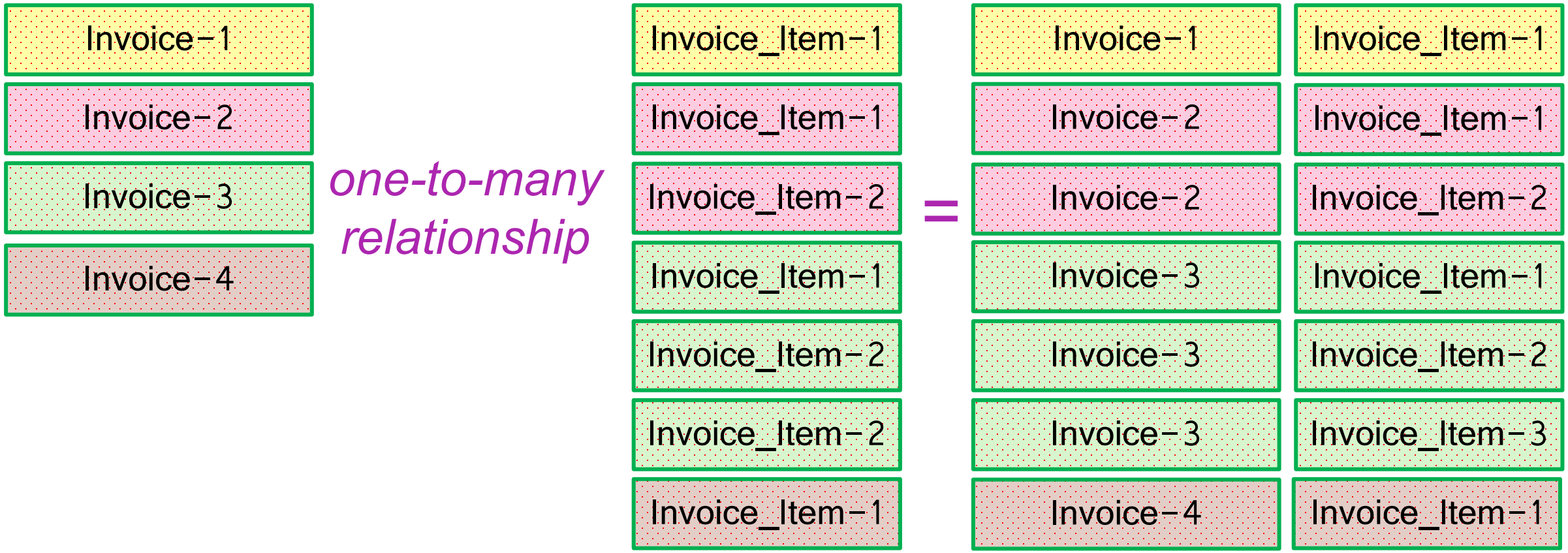
a *one-to-many* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which an element of ***R*** may be linked to many elements of ***S***, but a member of ***S*** is linked to only one element of ***R***.



one-to-many relationship

A *one-to-many* relationship between two tables means that a row in one table can have one or more row in the table on the other side of their relationship.

a *one-to-many* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which an element of ***R*** may be linked to many elements of ***S***, but a member of ***S*** is linked to only one element of ***R***.

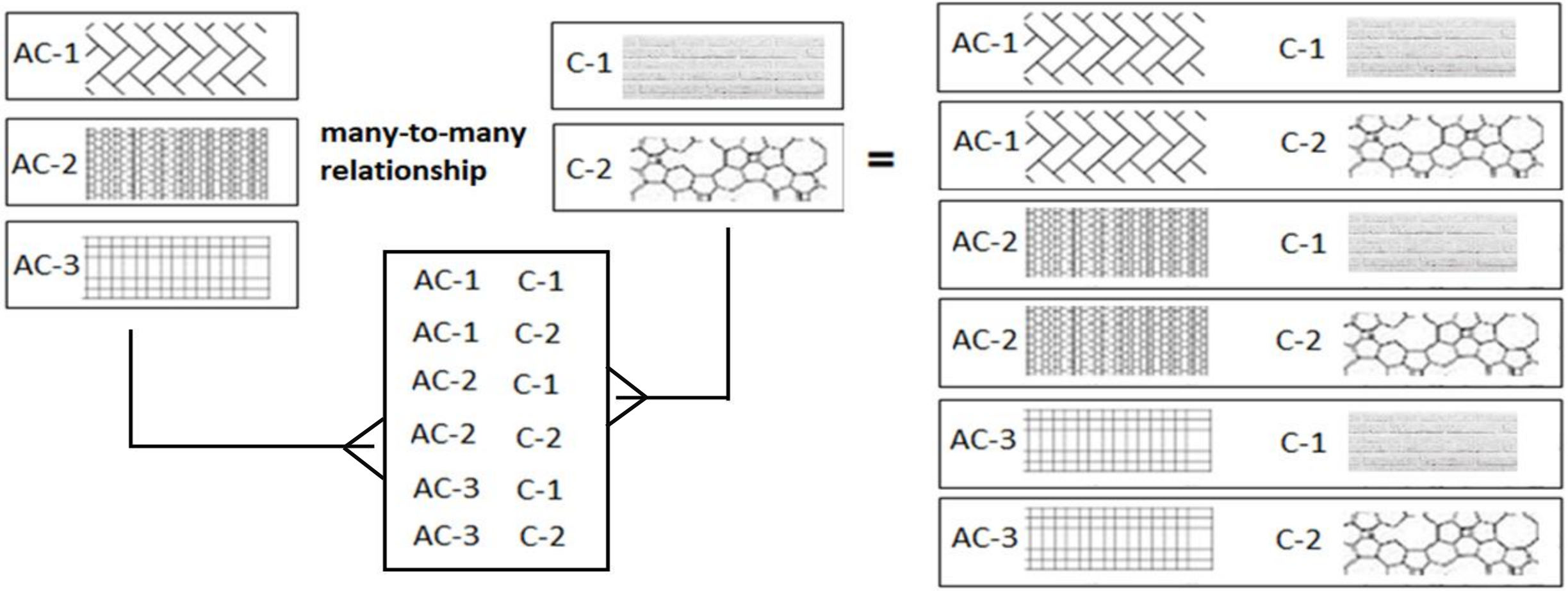


many-to-many relationship

One row in **Table A** can relate to **many rows in Table B**, and one row in **Table B** can also relate to **many rows in Table A**

many-to-many relationship

A *many-to-many* relationship is a type of cardinality that refers to the relationship between two entities ***R*** and ***S*** in which ***R*** may contain a parent instance for which there are many children in ***S*** and vice versa.

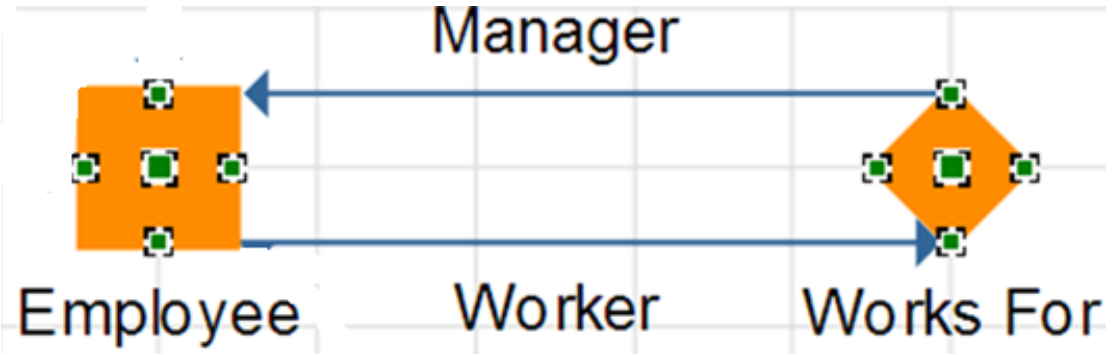


self-referencing relationship

An entity (table) is **related to itself** using a **foreign key** that points to its own primary key.

self-referencing relationship

A *"self-referencing"* or *"recursive"* relationship in databases or data structures means that a record within a table can reference another record in the same table.



Product Categories and Subcategories

CategoryID	CategoryName	ParentCategoryID
1	Electronics	NULL
2	Phones	1
3	Laptops	1
4	Smartphones	2
5	Gaming Laptops	3

MySQL is the most popular **Open Source** Relational Database Management System.

When you use MySQL, you're actually using at least two programmes. One program is the MySQL server (*mysqld.exe*) and other program is MySQL client program (*mysql.exe*) that connects to the database server.



What is SQL?

Remember:

what is sql?

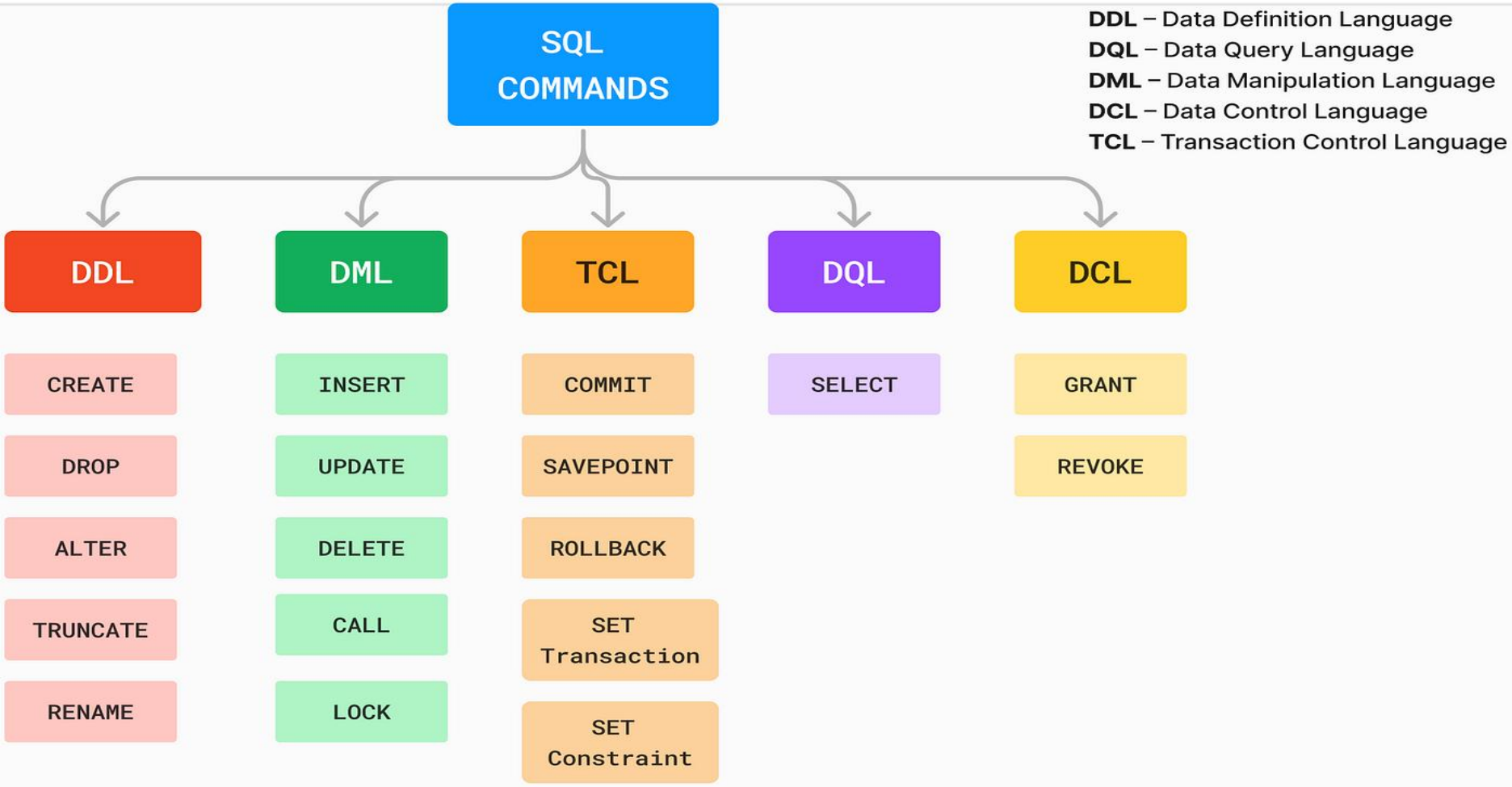
- **EXPLICIT or IMPLICIT commit will commit the data.**

SQL (Structured Query Language) is a database language designed and developed for managing data in relational database management systems (RDBMS). SQL is common language for all Relational Databases.



Remember:

- An ***implicit commit*** occurs ***automatically*** in MySQL ***without the need of COMMIT command***. This means changes made by the SQL statement are immediately saved to the database and ***cannot be rolled back***.
- An ***explicit commit*** is done by the user issuing a ***COMMIT*** command to ***manually save all changes*** made in the current transaction.



comments in mysql

- From a # character to the end of the line.
- From a -- sequence to the end of the line.
- From a /* sequence to the following */ sequence.

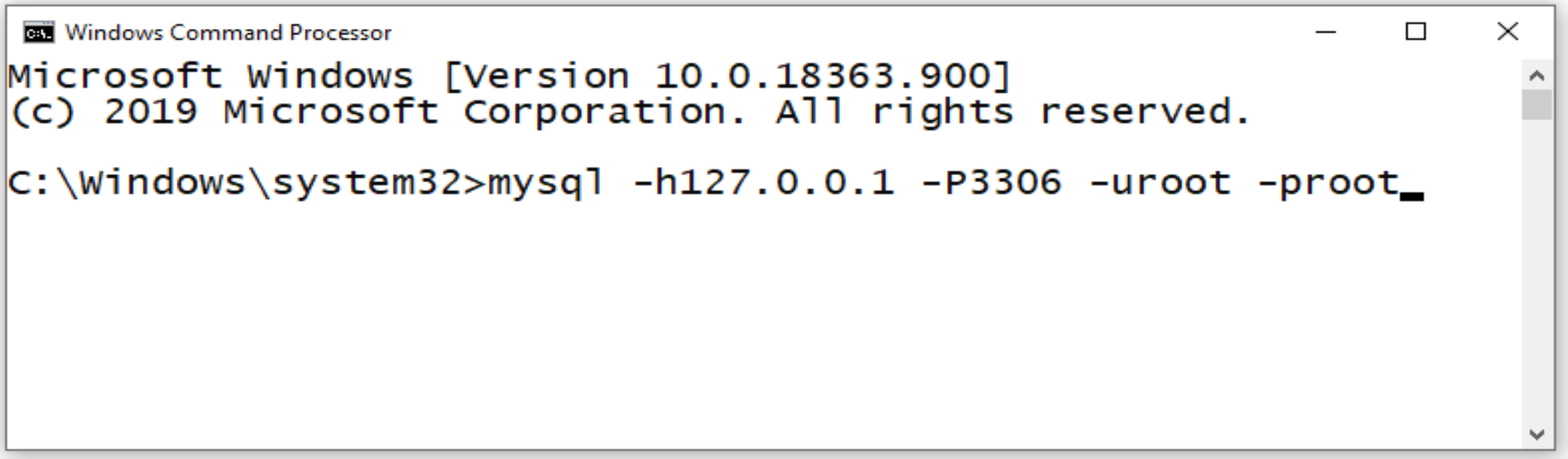
Reconnect to the server	**\r**
Execute a system shell command	**\!**
Exit mysql	**\q**
Change your mysql prompt.	**prompt str or \R str**

Login to MySQL

Default port for MySQL Server: 3306

login

- C:\> mysql -hlocalhost -P3307 -uroot -p
- C:\> mysql -h127.0.0.1 -P3307 -uroot -p [*database_name*]
- C:\> mysql -h192.168.100.14 -P3307 -uroot -psaleel [*database_name*]
- C:\> mysql --host localhost --port 3306 --user root --password=ROOT [*database_name*]
- C:\> mysql --host=localhost --port=3306 --user=root --password=ROOT [*database_name*]



- The **char** is a fixed-length character data type,
- The **varchar** is a variable-length character data type.

datatypes

ENAME CHAR (10)	S	A	L	E	E	L					LENGTH -> 10
ENAME VARCHAR2(10)	S	A	L	E	E	L					LENGTH -> 6

In MySQL

ENAME CHAR (10)	S	A	L	E	E	L					LENGTH -> 6
ENAME VARCHAR(10)	S	A	L	E	E	L					LENGTH -> 6

quantity VARCHAR(20) -- values: 20, '20', 'twenty'

datatype - string

Datatypes	Size	Description
CHAR [(length)]	0-255	***fixed-length character data type.***
VARCHAR (length)	0 to 65,535	***variable-length character data type.*** The maximum row size (65,535 bytes, which is shared among all columns.
TINYTEXT [(length)]	$(2^8 - 1)$ bytes	***variable-length string data type***
TEXT [(length)]	$(2^{16} - 1)$ bytes	65,535 bytes ~ 64kb
MEDIUMTEXT [(length)]	$(2^{24} - 1)$ bytes	16,777,215 bytes ~16MB
LONGTEXT [(length)]	$(2^{32} - 1)$ bytes	4,294,967,295 bytes ~4GB
ENUM('value1', 'value2',...)	65,535 distinct values	ENUM is a string data type that allows a column to **store only one value** from a predefined list of values.
SET('value1', 'value2',...)	64 distinct values	SET is a string data type that allows a column to store **zero, one, or multiple values** from a predefined list.
BINARY[(length)]	255	***fixed-length binary string data type***
VARBINARY(length)		***variable-length binary string data type***

- *SET sql_mode = '';*
- *SET sql_mode = 'PAD_CHAR_TO_FULL_LENGTH';*

- CREATE TABLE z1 (c1 ZEROFILL); // then Insert few records
- CREATE TABLE z1 (c1 INT(4) ZEROFILL);

datatype - numeric

Datatypes	Size	Description
TINYINT	1 byte	-128 to +127 **(The unsigned range is 0 to 255).**
SMALLINT [(length)]	2 bytes	-32768 to 32767. **(The unsigned range is 0 to 65535).**
MEDIUMINT [(length)]	3 bytes	-8388608 to 8388607. **(The unsigned range is 0 to 16777215).**
INT, INTEGER [(length)]	4 bytes	-2147483648 to 2147483647. **(The unsigned range is 0 to 4294967295).**
BIGINT [(length)]	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
FLOAT [(length[,decimals])]	4 bytes	**FLOAT(255,30)**
DOUBLE [PRECISION] [(length[,decimals])], REAL [(length[,decimals])]	8 bytes	**REAL(255,30) / DOUBLE(255,30)** REAL will get converted to DOUBLE
DECIMAL [(length[,decimals])], NUMERIC [(length[,decimals])]		**DECIMAL(65,30) / NUMERIC(65,30)** NUMERIC will get converted in DECIMAL

For: float(M,D), double(M,D) or decimal(M,D), M must be >= D

Here, **(M,D)** means than values can be stored with up to ***M*** digits in total, of which ***D*** digits may be after the decimal point.

TINYINT(1) ["TRUE" = 1 "FALSE" = 0]

UNSIGNED prohibits negative values.

datatype – date and time

Datatypes	**Size**	**Description**
YEAR	1 byte	YYYY ***(1901 to 2155)***
DATE	3 bytes	YYYY-MM-DD ***(1000-01-01 to 9999-12-31)***
TIME	3 bytes	HH:MM:SS
DATETIME	8 bytes	YYYY-MM-DD hh:mm:ss

datatype – boolean

A value of zero is considered false. Nonzero values are considered true.

ColumnName BOOLEAN

Note:

- BOOL and BOOLEAN are **synonym of TINYINT(1)**

Use a CREATE TABLE statement to specify the layout of your table.

Remember:

- Max 4096 columns per table provided the row size <= 65,535 Bytes.
- The NULL value is different from values such as 0 for numeric types or the empty string for string types.

create table

Use a **CREATE TABLE** statement to specify the layout of your table.

Note:

- **USER TABLES**: This is a collection of tables created and maintained by the user. Contain USER information.
- **DATA DICTIONARY**: This is a collection of tables created and maintained by the MySQL Server. It contains database information. All data dictionary tables are owned by the SYS user.

create table

syntax

CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
(*create_defineation*, . . .)
[table_options]
[partition_options]

create_definition:
col_name ***column_definition***

column_definition:
data_type [NOT NULL | NULL] [DEFAULT default_value]
[AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY]
[reference_definition]
| data_type [GENERATED ALWAYS] AS (expression) [VIRTUAL]
[VISIBLE | INVISIBLE]

table_options:
AUTO_INCREMENT = <number> // must be used with AUTO_INCREMENT definition
ENGINE [=] engine_name

e.g.

- CREATE TABLE student(
  ID INT,
  firstName VARCHAR(45),
  lastName VARCHAR(45),
  DoB DATE,
  emailID VARCHAR(128)
  );

show engines;
set default_storage_engine = memory;

insert rows

You can insert data using following methods:

- INSERT ... VALUES
- INSERT ... SET
- INSERT ... SELECT

dml- insert … values

INSERT inserts new row(s) into an existing table. The INSERT ... VALUES

INSERT [IGNORE] [INTO] tbl_name [PARTITION (*partition_name* [, *partition_name*] ...)] [(*field_name*, . . .)]
{ VALUES | VALUE } [ROW] ({ *expr* | *DEFAULT* }, . . .), [ROW] (. . .), [ROW] . . . [ON DUPLICATE KEY UPDATE
assignment_list]

The affected-rows value for an INSERT can be obtained using the ROW_COUNT() function.

```
INSERT INTO DEPT VALUES (1, 'HRD', 'Pune')
```

Column Values

```
INSERT INTO DEPT(ID, NAME, LOC) VALUES (1, 'HRD', 'Pune')
```

Column List

```
INSERT INTO DEPT(ID, NAME, LOC) VALUES (1, 'HRD', 'Baroda'),
 (2,'Sales','Surat'), (3,'Purchase','Pune'), (4,'Account','Mumbai')
```

Inserting multiple rows

md5()

- MD5() is a built-in function that generates a 128-bit hash value from a string.
- MD5 stands for **Message-Digest Algorithm** 5.
- It converts a string into a fixed-length 128-bit (16-byte) hash, normally represented as 32 hexadecimal characters.

MD5(*string*)

```
CREATE TABLE pwd (
    ID INT PRIMARY KEY,
    user_name VARCHAR(45),
    pwd VARCHAR(45)
);
```

```
INSERT INTO pwd(ID, user_name, password_hash) VALUES (1, 'saleel', MD5('p1'));
INSERT INTO pwd(ID, user_name, password_hash) VALUES (2, 'sharmin', MD5('p1'));
INSERT INTO pwd(ID, user_name, password_hash) VALUES (3, 'ruhan', MD5('p2'));
INSERT INTO pwd(ID, user_name, password_hash) VALUES (4, 'natasha', MD5('p2’));
```

```
SELECT * FROM pwd WHERE password_hash= MD5('p1');
```

```
SELECT MD5('S');
SELECT MD5('s');
SELECT MD5('saleel');
```

SHA2(*string, hash_length*)

str → text/value to hash
hash_length → 224, 256, 384, 512, or 0
0 means SHA-512

sha2

SHA2() is a function used to generate a SHA-2 family cryptographic hash from a string.

sha2()

INSERTing Sensitive Data Encryption

```
CREATE TABLE users
(
   user_id INT PRIMARY KEY,
   user_name VARCHAR(10),
   password_hash VARCHAR(255) GENERATED ALWAYS AS (SHA2(CONCAT(user_id, user_name), 256)) STORED
);
```

- ```
  INSERT INTO users(user_id, user_name)
  VALUES
    (1, 'saleel’),
    (2, 'sharmin’),
    (3, 'Ruhan’),
    (4, 'deep’),
    (5, 'neel’),
    (6, 'gau');
  ```

```
SELECT * FROM users WHERE password_hash = SHA2(CONCAT(user_id, user_name), 256);

SELECT * FROM users WHERE user_id = 1 AND password_hash = SHA2(CONCAT(user_id, user_name), 256);
```
```

IDENTITY is a synonym to the ***LAST_INSERT_ID*** variable.

auto_increment column

The **AUTO_INCREMENT** attribute can be used to generate a unique number/identity for new rows.

col_name data_type AUTO_INCREMENT [UNIQUE [KEY] | [PRIMARY] KEY]

generated column

virtual column - generated always

col_name data_type [GENERATED ALWAYS] AS (*expression*) | (CASE WHEN . . . THEN . . . ELSE . . . END)
[VIRTUAL | STORED]

- **VIRTUAL**: Column values are not stored, but are evaluated when rows are read, immediately after any BEFORE triggers. A virtual column takes no storage.
- **STORED**: Column values are evaluated and stored when rows are inserted or updated. A stored column does require storage space and can be indexed.

Note:

- The default is **VIRTUAL** if neither keyword is specified.

col_name data_type INVISIBLE

visible / invisible columns

Columns are visible by default. To explicitly specify visibility for a new column, use a VISIBLE or INVISIBLE keyword as part of the column definition for CREATE TABLE or ALTER TABLE.

col_name VARBINARY

varbinary column

VARBINARY is a variable-length binary string column. It's similar to VARCHAR, but it stores binary data, not text.

varbinary column

```
CREATE TABLE login (
  ID INT AUTO_INCREMENT PRIMARY KEY,
  userName VARCHAR(40),
  password VARBINARY(40) INVISIBLE
) ;
```

```
INSERT INTO login(userName, password) VALUES('ram', 'ram@123');
INSERT INTO login(userName, password) VALUES('pankaj', 'pankaj');
INSERT INTO login(userName, password) VALUES('rajan', 'rajan');
INSERT INTO login(userName, password) VALUES('ninad', 'ninad');
INSERT INTO login(userName, password) VALUES('omkar', 'omkar');
```

```
SELECT * FROM login;
SELECT username, CAST(password as CHAR) FROM login;
```

```
CREATE TABLE temp(
  c1 INT PRIMARY KEY,
  c2 VARBINARY(4) DEFAULT 0x89504E47);
```

a1 INT(10) zerofill	0	0	0	0	0	0	1	5	0	0	length(ID1) -> 10
a2 INT(10)							**3**	**6**	**7**	**5**	length(ID2) -> 4

zerofill value

When you select a column with the type ZEROFILL it pads the displayed value of the field with zeros up to the display width specified in the column definition.

Note:

- ZEROFILL attribute is use for numeric datatype.
- If you specify ZEROFILL for a column, MySQL automatically adds the ***UNSIGNED*** attribute to the column.

zerofill column

col_name data_type ZEROFILL

```
CREATE TABLE employee (
  ID INT ZEROFILL AUTO_INCREMENT PRIMARY KEY,
  firstName VARCHAR(40),
  salary INT,
  commission INT,
  total INT DEFAULT(salary + commission)
) ;
```

- INSERT INTO employee VALUES(NULL, 'ram', 4700, NULL, default);
- INSERT INTO employee VALUES(0, 'pankaj', 3400, 400 , default);
- INSERT INTO employee VALUES(100, 'rajan', 3200, NULL , default);
- INSERT INTO employee VALUES(NULL, 'ninad', 2600, 0, default);
- INSERT INTO employee VALUES(0, 'omkar', 4500, 300, default);
- INSERT INTO employee VALUES (-200, 'rahul', 3000, 300 , default);

- SELECT * FROM employee;
- SELECT ID, LENGTH(ID), salary, LENGTH(salary) FROM employee;

zerofill column

```
CREATE TABLE account (
  accountNumber INT ZEROFILL PRIMARY KEY,
  balance FLOAT,
  openingBalance FLOAT,
  accountName VARCHAR(45),
  custonerID INT,
  openingDate DATE
 );
```

Field	Type	Null	Key	Default	Extra
accountNumber	int(10) unsigned zerofill	NO	PRI	NULL	
balance	float	YES		NULL	
openingBalance	float	YES		NULL	
accountName	varchar(45)	YES		NULL	
custonerID	int	YES		NULL	
openingDate	date	YES		NULL	

int

Note:

- If you specify ZEROFILL for a numeric column, MySQL automatically adds the ***UNSIGNED*** attribute to the column.

Constraints define specific rules to the column(s) data in a database table. While ***inserting***, ***updating***, or ***deleting*** the data rows, if the rules of the constraint are **not followed**, the system will display an **error** message and ***the action will be terminated***. The SQL Constraints are defined while creating a new table. We can also **ALTER** the table and **ADD new** SQL Constraints. The MySQL Constraints are mainly used to maintain **data integrity**.

constraints

CONSTRAINT is used to define rules to allow or restrict what values can be stored in columns. The purpose of inducing constraints is to enforce the integrity of a database.

CONSTRAINTS can be classified into two types –

- *Column Level*
- *Table Level*

The column level constraints can apply only to one column where as table level constraints are applied to the entire table.

In MySQL, when you run SHOW INDEXES or SHOW CREATE TABLE, you may see MUL under the Key column.

Remember:

- PRI => primary key
- UNI => unique key
- MUL=> is basically an index that is neither a **primary key** nor a **unique key**. The name comes from "multiple" because multiple occurrences of the same value are allowed (Since it's not unique, the column can contain **duplicate values**.).

constraints

To limit or to restrict or to check or to control.

Note:

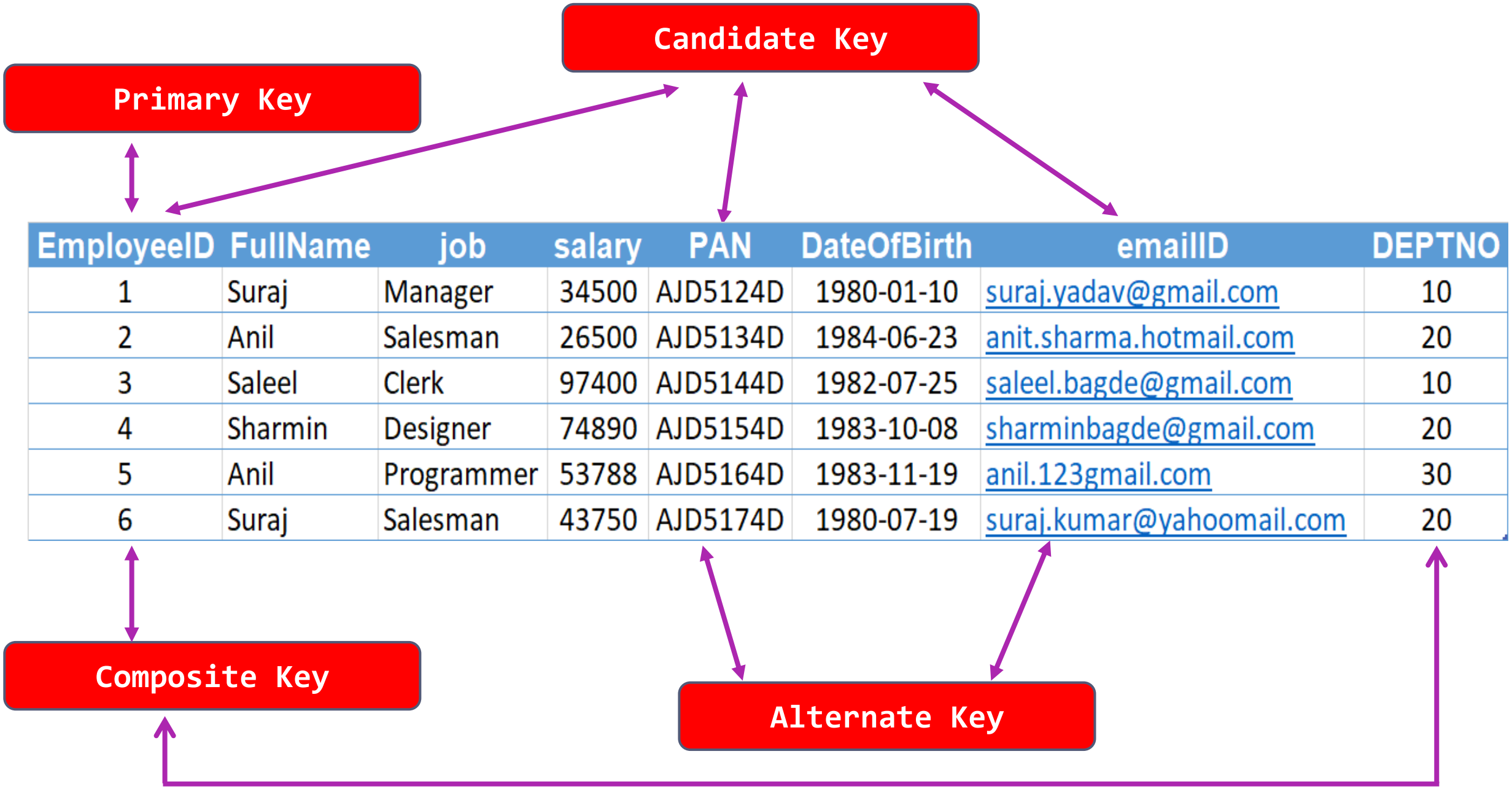
- a table with a foreign key that references another table's primary key is **MUL**.
- If more than one of the Key values applies to a given column of a table, Key displays the one with the highest priority, in the order **PRI**, **UNI**, and **MUL**.
- If a table has a PRIMARY KEY or UNIQUE NOT NULL index that consists of a single column that has an integer type, you can use **_rowid** to refer to the indexed column in SELECT statements.

Keys are used to establish relationships between tables and also to uniquely identify any record in the table.

types of Keys?

r = Employee(EmployeeID, FullName, job, salary, PAN, DateOfBirth, emailID, deptno)

- **Candidate Key:** are individual columns in a table that qualifies for uniqueness of all the rows. Here in Employee table EmployeeID, PAN or emailID are Candidate keys.
- **Primary Key**: is the columns you choose to maintain uniqueness in a table. Here in Employee table you can choose either EmployeeID, PAN or emailID columns, EmployeeID is preferable choice.
- **Alternate Key**: Candidate column other the primary key column, like if EmployeeID is primary key then , PAN or emailID columns would be the Alternate key.
- **Super Key**: If you add any other column to a primary key then it become a super key, like EmployeeID + FullName is a Super Key.
- **Composite Key**: If a table do not have any single column that qualifies for a Candidate key, then you have to select 2 or more columns to make a row unique. Like if there is no EmployeeID, PAN or emailID columns, then you can make FullName + DateOfBirth as Composite key. But still there can be a narrow chance of duplicate row.


Candidate Key
Primary Key
EmployeeID FullName job salary PAN DateOfBirth emailID DEPTNO
1 Suraj Manager 34500 AJD5124D 1980-01-10 suraj.yadav@gmail.com 10
2 Anil Salesman 26500 AJD5134D 1984-06-23 anit.sharma.hotmail.com 20
3 Saleel Clerk 97400 AJD5144D 1982-07-25 saleel.bagde@gmail.com 10
4 Sharmin Designer 74890 AJD5154D 1983-10-08 sharminbagde@gmail.com 20
5 Anil Programmer 53788 AJD5164D 1983-11-19 anil.123gmail.com 30
6 Suraj Salesman 43750 AJD5174D 1980-07-19 suraj.kumar@yahoomail.com 20
Composite Key
Alternate Key

No Primary Key in a Table

Anti-pattern

- Here,
```
CREATE TABLE orders (
    order_id INT,
    customer_name VARCHAR(100)
);
```

Why it's bad:

- Rows cannot be uniquely identified.
- Makes updates/deletes unreliable.

Better approach

- Here,
```
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_name VARCHAR(100)
);
```

Remember:

- A primary key cannot be NULL (absence of a value).
- A primary key value must be unique.
- A table has only one primary key.
- The primary key values cannot be changed, if it is referred by some other column.
- The primary key must be given a value when a new record is inserted.
- **An index can consist of 16 columns, at maximum. Since a PRIMARY KEY constraint automatically adds an index, it can't have more than 16 columns.**

Database	Max Columns in Primary Key
MySQL	16
PostgreSQL	32
Oracle	32
SQL Server	16
DB2	16
MariaDB	16

PRIMARY KEY constraint

A primary key is a special column (or set of combined columns) in a relational database table, that is used to uniquely identify each record. Each database table needs a primary key.

Note:

- Primary key in a relation is always associated with an **INDEX** object.
- If, we give on a column a combination of **NOT NULL & UNIQUE** key then it behaves like a PRIMARY key.
- If, we give on a column a combination of **UNIQUE key & AUTO_INCREMENT** then also it behaves like a PRIMARY key.
- Stability: The value of the primary key should be stable over time and not change frequently.

clustered and non-clustered index

Indexing in MySQL is a process that helps us to return the requested data from the table very fast. If the table does not have an index, it scans the whole table for the requested data.

MySQL allows two different types of Indexing:

- Clustered Index
- Non-Clustered Index

Clustered Index:- The InnoDB table uses a clustered index for optimizing the speed of most common lookups (SELECT statement) and DML operations like INSERT, UPDATE, and DELETE command. Clustered indexes sort and store the data rows in the table based on their key values that can be sorted in only one way. If the table column contains a primary key or unique key, MySQL creates a clustered index named PRIMARY based on that specific column.

Non-Clustered Index:- The indexes other than PRIMARY indexes (i.e. clustered indexes) called a non-clustered index. The non-clustered indexes are also known as secondary indexes. The non-clustered index and table data are both stored in different places. It is not sorted (ordering) the table data.

- CREATE TABLE test(c1 INT, c2 INT, c3 INT, c4 INT,c5 INT, c6 INT, c7 INT, c8 INT, c9 INT, c10 INT, c11 INT, c12 INT, c13 INT, c14 INT, c15 INT, c16 INT, c17 INT, c18 INT, c19 INT, c20 INT, PRIMARY KEY (c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11, c12, c13, c14, c15, c16, c17, c18)); // error

constraints – add primary key

col_name data_type PRIMARY KEY

The following example creates tables with **PRIMARY KEY** column.

- ```
  CREATE TABLE users(
    ID INT PRIMARY KEY,
    userName VARCHAR(25),
    password VARCHAR(25),
    email VARCHAR(255)
  ) ;
  ```

- ```
  CREATE TABLE purchase_orders(
    po_number INT,
    vendor_id INT NOT NULL,
    po_status INT NOT NULL,
    PRIMARY KEY(po_number)
  );
  ```

- ```
  CREATE TABLE person(
    ID INT NOT NULL UNIQUE,
    lastName VARCHAR(45),
    firstName VARCHAR(45),
    age INT,
    email VARCHAR(255)
  );
  ```

- ```
  CREATE TABLE supplier(
   supplier_id INT,
   supplier_name VARCHAR(50),
   contact_name VARCHAR(50),
   constraint pk_supplier_id PRIMARY KEY(supplier_id)
  );
  ```

constraints – add composite primary key

The following example creates tables with **COMPOSITE PRIMARY KEY** column.

- CREATE TABLE salesDetails (
  customerID INT,
  productID INT,
  timeID INT,
  qty INT,
  salesDate DATE,
  salesAmount INT,
  PRIMARY KEY(customerID , productID, timeID)
) ;

customerID	productID	timeID	quantity	salesDate	salesAmount
Cust-001	PRD-1	D1-T1	100	●●●●●●	25,00,000
Cust-001	PRD-1	D2-T1	100	●●●●●●	25,00,000
Cust-001	PRD-2	D1-T1	200	●●●●●●	50,00,000
Cust-002	PRD-1	D1-T1	100	●●●●●●	25,00,000
Cust-004	PRD-1	D1-T1	100	●●●●●●	25,00,000
Cust-004	PRD-2	D3-T1	200	●●●●●●	50,00,000

constraints – add composite primary key

The following example creates tables with **COMPOSITE PRIMARY KEY** column.

- ```
  CREATE TABLE payments (
    paymentID INT,
    orderID INT,
    amount INT,
    bankDetails VARCHAR(255),
    PRIMARY KEY(paymentID , orderID)
  ) ;
  ```

- ```
  CREATE TABLE order_product (
    orderID INT,
    productID INT,
    qty INT,
    rate INT,
    constraint pk_orderID_productID PRIMARY KEY(orderID, productID)
  ) ;
  ```

Try It

```
CREATE TABLE try(c1 INT, c2 INT, c3 INT, c4 INT, c5 INT, c6 INT
,c7 INT, c8 INT, c9 INT, c10 INT,c11 INT,c12 INT, c13 INT, c14 INT,
c15 INT, c16 INT,c17 INT, c18 INT, c19 INT, c20 INT, c21 INT, c22
INT, c23 INT, c24 INT, c25 INT, c26 INT, c27 INT, c28 INT, c29 INT ,
c30 INT, c31 INT, c32 INT, c33 INT, PRIMARY KEY(c1, c2, c3, c4,
c5, c6, c7, c8, c9, c10, c11, c12, c13, c14, c15, c16, c17));
```

Note:

- ERROR 1070 (42000): Too many key parts specified; max 16 parts allowed

```
ALTER TABLE table_name
  ADD [ CONSTRAINT constraint_name ]
    PRIMARY KEY (column1, column2, . . . column_n)
```

Add Primary Key using Alter

constraints – add primary key using alter

You can use the **ALTER TABLE** statement to **ADD PRIMARY KEY** on existing column.

```
ALTER TABLE table_name
 ADD [ CONSTRAINT constraint_name ]
  PRIMARY KEY (column1, column2, . . . column_n)
```

- ```
  CREATE TABLE vendors (
    vendor_id INT,
    vendor_name VARCHAR(25),
    address VARCHAR(255)
  );
  ```

- ```
  ALTER TABLE vendors ADD PRIMARY KEY(vendor_id);
  ```

- ```
  ALTER TABLE vendors ADD constraint pk_vendor_id PRIMARY KEY(vendor_id);
  ```
  ```

```
ALTER TABLE table_name
  DROP PRIMARY KEY
```

Drop Primary Key

constraints – drop primary key

You can use the **ALTER TABLE** statement to **DROP PRIMARY KEY**.

```
ALTER TABLE table_name
 DROP PRIMARY KEY
```

- ```
  CREATE TABLE vendors (
      vendor_id INT,
      vendor_name VARCHAR(25),
      address VARCHAR(255)
  );
  ```

```
ALTER TABLE vendors DROP PRIMARY KEY;
```
```

Remember:

- A unique key can be NULL (absence of a value).
- A unique key value must be unique.
- A table can have multiple unique key.
- A column can have unique key as well as a primary key.

UNIQUE KEY constraint

A **UNIQUE key** constraint is a set of one or more than one fields/columns of a table that uniquely identify a record in a database table.

Note:

- Unique key in a relation is always associated with an ***INDEX*** object.

constraints – add unique key

col_name data_type UNIQUE KEY

The following example creates table with **UNIQUE KEY** column.

- ```
  CREATE TABLE clients (
    client_id INT,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    company_name VARCHAR(255),
    email VARCHAR(255) UNIQUE
  );
  ```

- ```
  CREATE TABLE brands (
    ID INT,
    brandName VARCHAR(30),
    constraint uni_brandName UNIQUE(brandName)
  );
  ```

- ```
  SHOW  INDEX FROM clients;
  ```

- ```
  CREATE TABLE contacts (
    ID INT,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone VARCHAR(15),
    UNIQUE(phone)
  );
  ```

```
ALTER TABLE table_name
  ADD [ CONSTRAINT constraint_name ]
    UNIQUE (column1, column2, . . . column_n)
```

Add Unique Key using Alter

constraints – add unique key using alter

You can use the **ALTER TABLE** statement to **ADD UNIQUE KEY** on existing column.

```
ALTER TABLE table_name
 ADD [ CONSTRAINT constraint_name ]
  UNIQUE (column1, column2, . . . column_n)
```

- ```
  CREATE TABLE shop (
    ID INT,
    shop_name VARCHAR(30)
  );
  ```

- ```
  ALTER TABLE shop ADD UNIQUE(shop_name);
  ```
- ```
  ALTER TABLE shop ADD constraint uni_shop_name UNIQUE(shop_name);
  ```
  ```

```
ALTER TABLE table_name
  DROP INDEX constraint_name;
```

Drop Unique Key

constraints – drop unique key

You can use the **ALTER TABLE** statement to **DROP UNIQUE KEY**.

```
ALTER TABLE table_name
 DROP INDEX constraint_name;
```

- SELECT table_name, constraint_name, constraint_type FROM information_schema.table_constraints WHERE constraint_schema = 'z' AND table_name IN ('A', 'B');

- ALTER TABLE users DROP INDEX <COLUMN_NAME>;
- ALTER TABLE users DROP INDEX U_USER_ID; #CONSTRAINT NAME

-
```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255) UNIQUE KEY
) ;
```

-
```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  constraint uni_email UNIQUE KEY(email)
) ;
```

- ALTER TABLE users DROP INDEX email;

- ALTER TABLE users DROP INDEX uni_email;

[CONSTRAINT [*symbol*]] FOREIGN KEY (*col_name*, . . .) REFERENCES *tbl_name* (*col_name*, . . .)

[ON DELETE CASCADE | SET NULL]
[ON UPDATE CASCADE | SET NULL]

FOREIGN KEY constraint

A **FOREIGN KEY** is a **key** used to link two tables together. A **FOREIGN KEY** is a field (or collection of fields) in one table that refers to the PRIMARY **KEY** in another table. The table containing the **foreign key** is called the child table, and the table containing the candidate **key** is called the referenced or parent table.

constraints – foreign key

Remember:

- A foreign key can have a different column name from its primary key.
- DataType of primary key and foreign key column must be same.
- It ensures rows in one table have corresponding rows in another.
- Unlike the Primary key, they do not have to be unique.
- Foreign keys can be null even though primary keys can not.

Note:

- The table containing the FOREIGN KEY is referred to as the child table, and the table containing the PRIMARY KEY (referenced key) is the parent table.
- PARENT and CHILD tables must use the same storage engine,
- and they cannot be defined as temporary tables.

insert, update, & delete – (primary key/foreign key)

A referential constraint could be violated in following cases.

- An **INSERT** attempt to add a row to a child table that has a value in its foreign key columns that does not match a value in the corresponding parent table's column.
- An **UPDATE** attempt to change the value in a child table's foreign key columns to a value that has no matching value in the corresponding parent table's parent key.
- An **UPDATE** attempt to change the value in a parent table's parent key to a value that does not have a matching value in a child table's foreign key columns.
- A **DELETE** attempt to remove a record from a parent table that has a matching value in a child table's foreign key columns.

Note:

- PARENT and CHILD tables must use the same storage engine,
- and they cannot be defined as temporary tables.
- If we don't give constraint name. System will automatically generated the constraint name and will assign to foreign key constraint. **e.g. login_ibfk_1, login_ibfk_2, …..**

Remember:

anomaly – (primary key/foreign key)

Student (parent) Table

RollNo	Name	Mobile	City	State	isActive
1	Ramesh	●●●●	Pune	MH	1
2	Amit	●●●●	Baroda	GJ	1
3	Rajan	●●●●	Surat	GJ	1
4	Bhavin	●●●●	Baroda	GJ	1
5	Pankaj	●●●●	Surat	GJ	1

student_course (child) Table

RollNo	CourceDuration	CourceName
1	1.5 month	RDBMS
2	1.2 month	NoSQL
3	2 month	Networking
1	2 month	Java
2	2 month	.NET

Insertion anomaly:

- If we try to insert a record in Student_Course (child) table with RollNo = 7, it will not allow.

Updation and Deletion anomaly:

- If you try to chance the RollNo from Student (parent) table with RollNo = 6 whose RollNo = 1 , it will not allow.
- If you try to chance the RollNo from Student_Course (child) table with RollNo = 9 whose RollNo = 3 , it will not allow.
- If we try to delete a record from Student (parent) table with RollNo = 1, it will not allow.

alter, drop – (primary key/foreign key)

Remember:

Parent Table

```
student  = {
   rollno INT,  * (PK)
   name VARCHAR(10),
   mobile VARCHAR(10),
   city VARCHAR(10),
   state VARCHAR(10),
   isActive BOOL
}
```

Child Table

```
student_course = {
   rollno INT,   * (FK)
   courceduration VARCHAR(10),
   courcename VARCHAR(10)
}
```

DDL command could be violated in following cases.

Alter command:

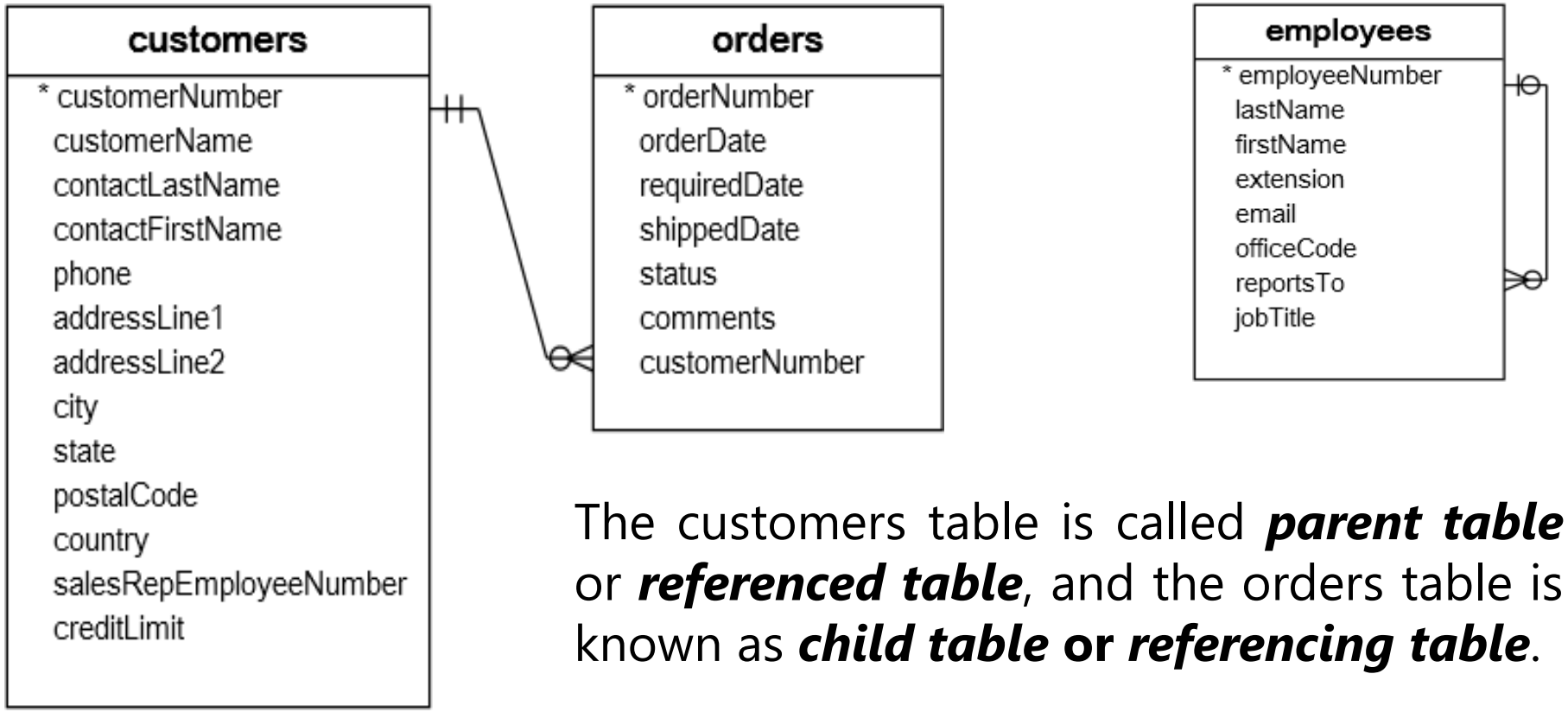
- If we try to modify datatype of RollNo in Student or Student_Course table with VARCHAR, it will not allow.
- If we try to apply auto_increment to RollNo in Student table, it will not allow
- If we try to drop RollNo column from Student table , it will not allow.

Drop command:

- If we try to drop Student (parent) table, it will not allow.

constraints – foreign key

A foreign key is a field in a table that matches another field of another table. A foreign key places constraints on data in the related tables, which enables MySQL to maintain referential integrity.



The customers table is called ***parent table*** or ***referenced table***, and the orders table is known as ***child table*** **or** ***referencing table***.

constraints – foreign key

- ```
  CREATE TABLE department(
    department_id INT UNSIGNED  PRIMARY KEY
  );
  ```

```
CREATE TABLE employee(
  department_id INT,
  FOREIGN KEY (department_id)  REFERENCES
   department(department_id)
);
```

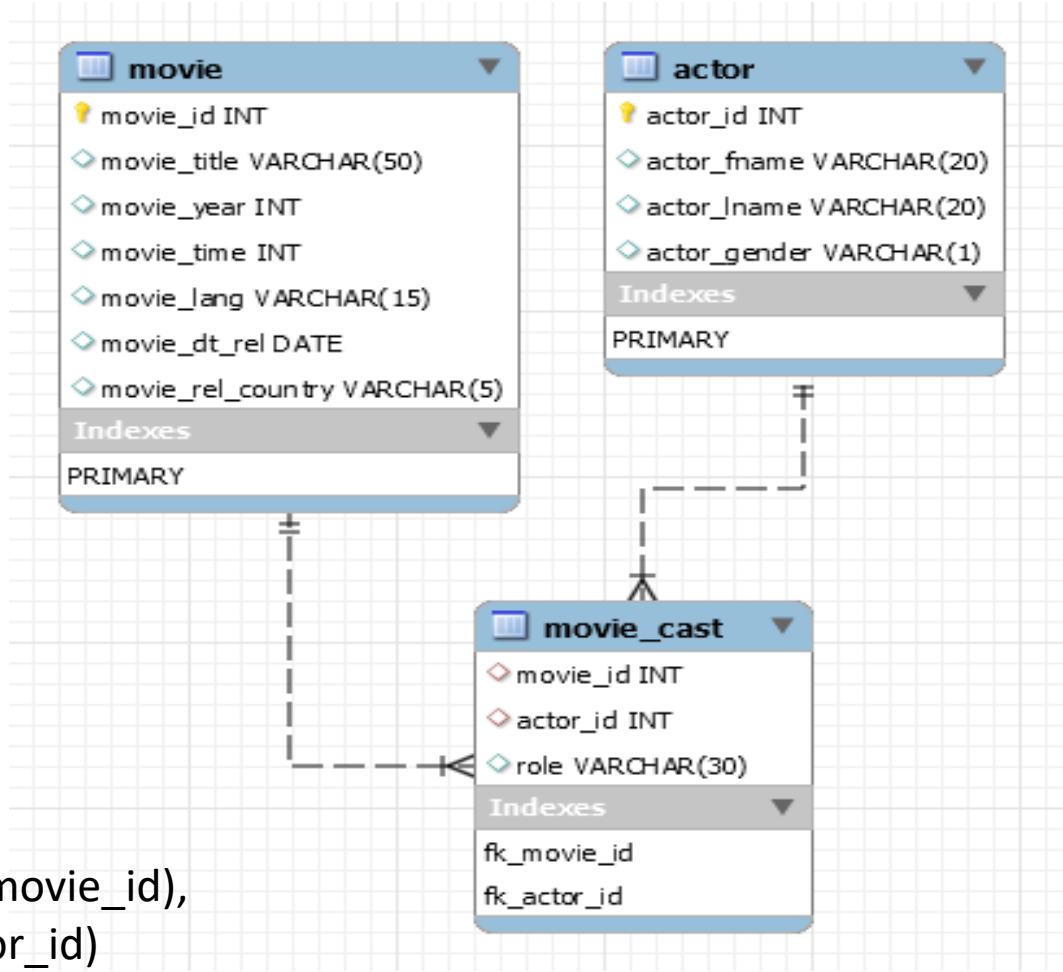
```
CREATE TABLE employee(
  department_id INT UNSIGNED,
  FOREIGN KEY (department_id)  REFERENCES
   department(department_id)
);
```
```

constraints – foreign key

A foreign key is a field in a table that matches another field of another table. A foreign key places constraints on data in the related tables, which enables MySQL to maintain referential integrity.

- CREATE TABLE movie (
  movie_id INT PRIMARY KEY,
  movie_title VARCHAR(50),
  movie_year INT,
  movie_time INT,
  movie_lang VARCHAR(15),
  movie_dt_rel DATE,
  movie_rel_country VARCHAR(5)
  );

- CREATE TABLE actor (
  actor_id INT PRIMARY KEY,
  actor_fname VARCHAR(20),
  actor_lname VARCHAR(20),
  actor_gender VARCHAR(1)
  );



- CREATE TABLE movie_cast (
  movie_id INT,
  actor_id INT,
  role VARCHAR(30),
  constraint fk_movie_id FOREIGN KEY(movie_id) REFERENCES movie(movie_id),
  constraint fk_actor_id FOREIGN KEY(actor_id) REFERENCES actor(actor_id)
  );

QUESTION – find foreign key columns

The following example find ***Foreign Key*** columns.

- ```
  CREATE TABLE owner (
    owner_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(255)
  );
  ```

- ```
  CREATE TABLE shop (
    shop_id INT,
    owner_id INT,
    shop_name VARCHAR(30)
  );
  ```

- ```
  CREATE TABLE brands (
    brand_id INT PRIMARY KEY,
    brand_name VARCHAR(30) UNIQUE
   );
  ```

- ```
  CREATE TABLE contacts (
    contact_id INT PRIMARY KEY,
    owner_id INT,
    contact_number VARCHAR(15)
  );
  ```

- ```
  CREATE TABLE shop_brand (
    ID INT PRIMARY KEY,
    shop_id INT,
    brand_id INT
  );
  ```
  ```

```
ALTER TABLE table_name
  ADD [ CONSTRAINT constraint_name ]
    FOREIGN KEY (child_col1, child_col2, . . . child_col_n)
    REFERENCES parent_table (parent_col1, parent_col2, . . . parent_col_n);
```

Add Foreign Key Constraint using Alter

constraints – add foreign key using alter

You can use the **ALTER TABLE** statement to **ADD FOREIGN KEY** on existing column.

```
ALTER TABLE table_name
 ADD [ CONSTRAINT constraint_name ]
   FOREIGN KEY (child_col1, child_col2, . . . child_col_n)
   REFERENCES parent_table (parent_col1, parent_col2, . . . parent_col_n);
```

```
CREATE TABLE users (
   ID INT PRIMARY KEY,
   userName VARCHAR(40),
   password VARCHAR(255),
   email VARCHAR(255) UNIQUE KEY
) ;
```

```
CREATE TABLE login (
   ID INT PRIMARY KEY,
   userID INT,
   loginDate DATE,
   loginTime TIME
) ;
```

- ALTER TABLE login ADD FOREIGN KEY(userID) REFERENCES users(ID);
- ALTER TABLE login ADD constraint fk_userID FOREIGN KEY(userID) REFERENCES users(ID);

```
ALTER TABLE table_name
  DROP  FOREIGN KEY constraint_name
```

Drop Foreign Key Constraint using Alter

constraints – drop foreign key

You can use the **ALTER TABLE** statement to **DROP FOREIGN KEY**.

- ```
  CREATE TABLE users (
    ID INT PRIMARY KEY ,
    userName VARCHAR(40),
    password VARCHAR(255),
    email VARCHAR(255)
  ) ;
  ```

- ```
  CREATE TABLE login (
    ID INT PRIMARY KEY,
    userID INT,
    loginDate DATE,
    loginTime TIME,
    FOREIGN KEY(userID) REFERENCES users(ID)
  ) ;
  ```

- ```
  CREATE TABLE login (
    ID INT PRIMARY KEY,
    userID INT,
    loginDate DATE,
    loginTime TIME,
    constraint fk_userID FOREIGN KEY(userID)  REFERENCES users(ID)
  ) ;
  ```

- ```
  ALTER TABLE login DROP FOREIGN KEY fk_userID;
  ```

- ```
  ALTER TABLE login DROP FOREIGN KEY login_ibfk_1; // login_ibfk_1 is the default constraint name.
  ```

- ```
  SELECT table_name, constraint_name, constraint_type FROM information_schema.table_constraints WHERE
  table_schema = 'DB2';
  ```

constraints – foreign key

- [ON DELETE *reference_option*]
- [ON UPDATE *reference_option*]

reference_option: RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT

Cascaded FOREIGN KEY actions do not activate triggers.

Remember:

- When an UPDATE or DELETE operation affects a key value in the parent table that has matching rows in the child table, the result depends on the referential action specified using ON UPDATE and ON DELETE sub clauses of the FOREIGN KEY clause.

- ON DELETE or ON UPDATE that is not specified, the default action is always RESTRICT.

constraints – foreign key

Remember:

- CASCADE: DELETE or UPDATE the row from the parent table, and automatically DELETE or UPDATE the matching rows in the child table. Both ON DELETE CASCADE and ON UPDATE CASCADE are supported.
- SET NULL: DELETE or UPDATE the row from the parent table, and set the foreign key column or columns in the child table to NULL. Both ON DELETE SET NULL and ON UPDATE SET NULL clauses are supported.

```
CREATE TABLE users (
  ID INT PRIMARY KEY ,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255)
);
```

```
CREATE TABLE login (
  ID INT PRIMARY KEY,
  userID INT,
  loginDate DATE,
  loginTime TIME,
  FOREIGN KEY(userID)  REFERENCES users(ID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
CREATE TABLE login (
  ID INT PRIMARY KEY,
  userID INT,
  loginDate DATE,
  loginTime TIME,
  FOREIGN KEY(userID)  REFERENCES users(ID) ON DELETE SET NULL ON UPDATE SET NULL
);
```

on delete / on update – foreign key

SET NULL: Delete or update the row from the parent table and set the foreign key column or columns in the child table to NULL. If you specify a SET NULL action, make sure that you have not declared the columns in the child table as NOT NULL.

RESTRICT: Rejects the delete or update operation for the parent table. Specifying RESTRICT (or NO ACTION) is the same as omitting the ON DELETE or ON UPDATE clause.

NO ACTION: Is equivalent to RESTRICT. The MySQL Server rejects the delete or update operation for the parent table if there is a related foreign key value in the referenced table.

SET DEFAULT: This action is recognized by the MySQL parser, but both InnoDB and NDB reject table definitions containing ON DELETE SET DEFAULT or ON UPDATE SET DEFAULT clauses.

- `CREATE TABLE test (c1 INT, c2 INT, c3 INT, check (c3 = SUM(c1)));`

```
// ERROR
SUM(SAL) MIN(SAL) COUNT(*)
AVG(SAL) MAX(SAL) COUNT(JOB)
```

Check Constraint

operators like =, >, <, BETWEEN, IN, LIKE. Can use.

constraints – check

CHECK condition expressions must follow some rules.

- Literals, deterministic built-in functions, and operators are permitted.
- Non-generated and generated columns are permitted, except columns with the AUTO_INCREMENT attribute.
- Sub-queries are not permitted.
- Environmental variables (such as CURRENT_USER, CURRENT_DATE, …) are not permitted.
- Non-Deterministic built-in functions (such as AVG, COUNT, RAND, LAST_INSERT_ID, FIRST_VALUE, LAST_VALUE, ...) are not permitted.
- Variables (system variables, user-defined variables, and stored program local variables) are not permitted.
- Stored functions and user-defined functions are not permitted.

Note:

Prior to MySQL 8.0.16, CREATE TABLE permits only the following limited version of table CHECK constraint syntax, which is parsed and ignored.

Remember:

If you omit the constraint name, MySQL automatically generates a name with the following convention:

- table_name_chk_n

constraints – check

col_name data_type CHECK(*expr*)

The following example creates **USERS** table with **CHECK** column.

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  ratings INT CHECK(ratings > 50)
);
```

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  ratings INT,
  CHECK(ratings > 50)
);
```

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  ratings INT,
  constraint chk_ratings CHECK(ratings > 50)
);
```

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  ratings INT,
  constraint chk_ratings  CHECK(ratings > 50),
  constraint chk_email CHECK(LENGTH(email) > 12)
);
```

constraints – check

col_name data_type CHECK(*expr*)

The following example creates **USERS** table with **CHECK** column.

```
CREATE TABLE users (
   ID INT PRIMARY KEY,
   startDate DATE,
   endDate DATE,
  constraint chk_endDate CHECK(endDate > startDate +  INTERVAL 7 day)
);
```

- SELECT * FROM check_constraints WHERE CONSTRAINT_SCHEMA = 'z';

```
ALTER TABLE table_name
  ADD   [ CONSTRAINT constraint_name ]
    CHECK (conidiation)
```

Add Check Constraint using Alter

constraints – add check using alter

You can use the **ALTER TABLE** statement to **ADD CHECK KEY** on existing column.

```
ALTER TABLE table_name
 ADD  CONSTRAINT [ constraint_name ]
  CHECK (conidiation)
```

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  ratings INT
) ;
```

- ALTER TABLE users ADD CHECK(ratings > 50);
- ALTER TABLE users ADD constraint chk_ratings CHECK(ratings > 50);

```
ALTER TABLE table_name
  DROP { CHECK | CONSTRAINT } constraint_name
```

drop check constraint

constraints – drop check key

You can use the **ALTER TABLE** statement to **DROP CHECK KEY**.

```
ALTER TABLE table_name
 DROP { CHECK | CONSTRAINT } constraint_name
```

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  ratings INT,
  constraint chk_ratings CHECK(ratings > 50)
) ;
```

- ALTER TABLE users DROP CHECK chk_ratings;
- ALTER TABLE users DROP constraint chk_ratings;
- ALTER TABLE users DROP CHECK users_chk_1;

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255),
  email VARCHAR(255),
  ratings INT,
  CHECK(ratings > 50)
) ;
```

- SELECT table_name, constraint_name, constraint_type FROM information_schema.table_constraints WHERE table_schema = 'DB2' AND (table_name LIKE 'U%' OR table_name LIKE 'L%');

The check constraint defined on a table must refer to only columns in that table. It can not refer to columns in other tables.

- `CREATE TABLE test (CHECK(c3 > (c1 + c2)), c1 INT, c2 INT, c3 INT);`
- `CREATE TABLE test (c1 INT, c2 INT, c3 INT, CHECK(c3 > (c1 + c2)));`
- `CREATE TABLE test (CHECK(c3 > (c1 + c2)), PRIMARY KEY(c1), c1 INT, c2 INT, c3 INT);`
- `CREATE TABLE test (a INT CHECK (a >= 0),b INT  CHECK (b >= 0), CHECK ( a + b <= 10));`

- `ALTER TABLE test ADD constraint chk_id CHECK(ID > 10);`
- `ALTER TABLE test DROP CHECK chk_id;`

- ```
  CREATE TABLE payments(
    paymentID INT PRIMARY KEY,
    paymentMode VARCHAR(20),
   CHECK (paymentMode IN ('Cash', 'Card', 'UPI', 'Insurance'))
  );
  ```
  ```

check with (in, like, and between)

The **CHECK** constraint using IN, LIKE, and BETWEEN.

```
CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(255) CHECK(LENGTH(password) > 5),
  email VARCHAR(255),
  country VARCHAR(255) CHECK(country LIKE ('I%') OR country LIKE ('U%')),
  ratings INT CHECK(ratings BETWEEN 1 AND 5 OR ratings BETWEEN 12 AND 25),
  isActive BOOL CHECK(isActive IN (1, 0)),
  startDate DATE,
  endDate DATE,
  constraint chk_endDate CHECK(endDate > startDate +  INTERVAL 7 day)
) ;
```

alter table

ALTER TABLE changes the structure of a table.

Note:

- you can add or delete columns,
- create or destroy indexes,
- change the type of existing columns, or
- rename columns or the table itself.
- You cannot change the position of columns in table structure. If not, then what? create a new table with SELECT statement.

syntax

alter table

ALTER TABLE tbl_name

[*alter_specification* [, *alter_specification*] . . .

- | ADD [COLUMN] *col_name column_definition* [FIRST | AFTER *col_name*]
- | ADD [COLUMN] (*col_name column_definition*, . . .)
- | ADD {INDEX|KEY} [index_name] (*index_col_name*, . . .)
- | ADD [CONSTRAINT [*symbol*]] PRIMARY KEY
- | ADD [CONSTRAINT [*symbol*]] UNIQUE KEY
- | ADD [CONSTRAINT [*symbol*]] FOREIGN KEY *reference_definition*
- | CHANGE [COLUMN] *old_col_name new_col_name* column_definition [FIRST|AFTER *col_name*]
- | MODIFY [COLUMN] *col_name column_definition* [FIRST | AFTER *col_name*]
- | DROP [COLUMN] *col_name*
- | DROP PRIMARY KEY
- | DROP {INDEX|KEY} *index_name*
- | DROP FOREIGN KEY *fk_symbol*
- | RENAME [TO|AS] *new_tbl_name*
- | RENAME COLUMN *old_col_name* TO *new_col_name*
- | ALTER [COLUMN] *col_name* { SET DEFAULT {*literal* | (*expr*)} | DROP DEFAULT } | SET { VISIBLE | INVISIBLE }

Remember:

- **Change Columns** :- You can rename a column using a CHANGE old_col_name new_col_name column_definition clause. To do so, specify the old and new column names and the definition that the column currently has.
- **Modify Columns :-** You can also use MODIFY to change a column's type without renaming it.
- **Dropping Columns :-** If a table contains only one column, the column cannot be dropped. If columns are dropped from a table, the columns are also removed from any index of which they are a part. If all columns that make up an index are dropped, the index is dropped as well.

Note:

- To convert a table from one storage engine to another, use an ALTER TABLE statement that indicates the new engine:

```
ALTER TABLE tbl_name ENGINE = InnoDB;

ALTER TABLE tbl_name ADD col1 INT, ADD col2 INT;

ALTER TABLE tbl_name DROP COLUMN col1, DROP COLUMN col2 , ADD col3 INT;
```

add column

ALTER TABLE tbl_name [alter_specification [, alter_specification] . . .]

alter_specification

- ADD [COLUMN] col_name column_definition [FIRST | AFTER col_name]
- ADD [COLUMN] (col_name column_definition, . . .)

add column

```
CREATE TABLE vehicles(
    vehicleID INT PRIMARY KEY ,
    year INT,
    make VARCHAR(100)
);
```

Field	Type	Null	Key	Default	Extra
vehicleID	int	NO	PRI	NULL	
year	int	YES		NULL	
make	varchar(100)	YES		NULL	

```
INSERT INTO vehicles VALUES (111, 2000, 'Honda');
INSERT INTO vehicles VALUES (112, 2002, 'Hyundai');
INSERT INTO vehicles VALUES (113, 2000, 'Jeep');
INSERT INTO vehicles VALUES (114, 2005, 'Toyota');
```

```
ALTER TABLE vehicles
    ADD ID INT UNIQUE auto_increment first,
    ADD model VARCHAR(100) NOT NULL,
    ADD color VARCHAR(50),
    ADD note VARCHAR(255);
```

Field	Type	Null	Key	Default	Extra
ID	int	NO	UNI	NULL	auto_increment
vehicleID	int	NO	PRI	NULL	
year	int	YES		NULL	
make	varchar(100)	YES		NULL	
model	varchar(100)	NO		NULL	
color	varchar(50)	YES		NULL	
note	varchar(255)	YES		NULL	

ID	vehicleID	year	make	model	color	note
1	111	2000	Honda		NULL	NULL
2	112	2002	Hyundai		NULL	NULL
3	113	2000	Jeep		NULL	NULL
4	114	2005	Toyota		NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL

modify column

ALTER TABLE tbl_name [alter_specification [, alter_specification] ...]

alter_specification

- MODIFY [COLUMN] col_name column_definition [FIRST | AFTER col_name]

modify column

```
CREATE TABLE vehicles(
    vehicleID INT PRIMARY KEY ,
    year INT,
    make VARCHAR(100),
    model VARCHAR(100) NOT NULL,
    color VARCHAR(50),
    note VARCHAR(255)
);
```

Field	Type	Null	Key	Default	Extra
vehicleID	int	NO	PRI	NULL	
year	int	YES		NULL	
make	varchar(100)	YES		NULL	
model	varchar(100)	NO		NULL	
color	varchar(50)	YES		NULL	
note	varchar(255)	YES		NULL	

```
ALTER TABLE vehicles
    MODIFY year SMALLINT NOT NULL,
    MODIFY make VARCHAR(150) NOT NULL,
    MODIFY color VARCHAR(20) NOT NULL;
```

Field	Type	Null	Key	Default	Extra
vehicleID	int	NO	PRI	NULL	
year	smallint	NO		NULL	
make	varchar(150)	NO		NULL	
model	varchar(100)	NO		NULL	
color	varchar(20)	NO		NULL	
note	varchar(255)	YES		NULL	

```
INSERT INTO vehicles VALUES (111, 2000, 'Honda', 'A1', 'silver', ' Honda was the first Japanese automobile manufacturer to release a dedicated luxury brand, Acura, in 1986.');
```

```
INSERT INTO vehicles VALUES (112, 2002, 'Hyundai', 'AC1', 'white', ' Hyundai operates the world's largest integrated automobile manufacturing facility in Ulsan, South Korea which has an annual production capacity of 1.6 million units.');
```

```
INSERT INTO vehicles VALUES (113, 2000, 'Jeep', 'D2', 'black', ' Fiat Chrysler Automobiles has owned Jeep since 2014. Previous owners include the Kaiser Jeep Corporation and American Motors Corporation. Most Jeeps are American-made, except for a select few models. The Toledo Assembly Complex in Ohio manufactures the Jeep Wrangler.');
```

rename column

ALTER TABLE tbl_name [alter_specification [, alter_specification] ...]

alter_specification

- RENAME COLUMN old_col_name TO new_col_name

rename column

- CREATE TABLE vehicles (
  vehicleID INT,
  year SMALLINT,
  make VARCHAR(150),
  model VARCHAR(100),
  color VARCHAR(20),
  note VARCHAR(255)
  );

Field	Type	Null	Key	Default	Extra
vehicleID	int	YES		NULL	
year	smallint	YES		NULL	
make	varchar(150)	YES		NULL	
model	varchar(100)	YES		NULL	
color	varchar(20)	YES		NULL	
note	varchar(255)	YES		NULL	

- ALTER TABLE vehicles
  RENAME COLUMN year TO model_year

Field	Type	Null	Key	Default	Extra
vehicleID	int	YES		NULL	
model_year	int	NO		NULL	
make	varchar(150)	YES		NULL	
model	varchar(100)	YES		NULL	
model_color	varchar(20)	YES		NULL	
vehicleCondition	varchar(150)	YES		NULL	

change column

ALTER TABLE tbl_name [alter_specification [, alter_specification] ...]

alter_specification

- CHANGE [COLUMN] old_col_name new_col_name column_definition [FIRST | AFTER col_name]

change column

- CREATE TABLE vehicles (
  vehicleID INT,
  year SMALLINT,
  make VARCHAR(150),
  model VARCHAR(100),
  color VARCHAR(20),
  note VARCHAR(255)
  );

Field	Type	Null	Key	Default	Extra
vehicleID	int	YES		NULL	
year	smallint	YES		NULL	
make	varchar(150)	YES		NULL	
model	varchar(100)	YES		NULL	
color	varchar(20)	YES		NULL	
note	varchar(255)	YES		NULL	

- ALTER TABLE vehicles
  CHANGE year model_year INT,
  CHANGE color model_color VARCHAR(20),
  CHANGE note vehicleCondition VARCHAR(150);

Field	Type	Null	Key	Default	Extra
vehicleID	int	YES		NULL	
model_year	int	NO		NULL	
make	varchar(150)	YES		NULL	
model	varchar(100)	YES		NULL	
model_color	varchar(20)	YES		NULL	
vehicleCondition	varchar(150)	YES		NULL	

change column

- CREATE TABLE users (
  ID INT PRIMARY KEY,
  userName VARCHAR(40),
  password VARCHAR(25),
  email VARCHAR(255)
  );

- CREATE TABLE login (
  ID INT PRIMARY KEY,
  userID INT,
  loginDate DATE,
  loginTime TIME,
  constraint fk_userID FOREIGN KEY(userID) REFERENCES users(ID)
) ;

- INSERT INTO users VALUES (1, 'rajan', 'ranaj123', 'rajan447.gmail.com');
- INSERT INTO users VALUES (2, 'raj', 'raj', 'raj.gmail.com');

- INSERT INTO login VALUES (1, 1, curdate(), curtime());
- INSERT INTO login VALUES (2, 1, curdate(), curtime());
- INSERT INTO login VALUES (3, 2, curdate(), curtime());
- INSERT INTO login VALUES (4, NULL, curdate(), curtime());

- ALTER TABLE users CHANGE ID userID INT;
- ALTER TABLE login CHANGE userID UID INT;

- INSERT INTO login VALUES (5, NULL, curdate(), curtime());

drop column

ALTER TABLE tbl_name [*alter_specification* [, *alter_specification*] . . .]

alter_specification

- DROP [COLUMN] *col_name*

drop column

- CREATE TABLE vehicles(
  vehicleID INT,
  model_year SMALLINT,
  make VARCHAR(150),
  model VARCHAR(100),
  model_color VARCHAR(20),
  vehicleCondition VARCHAR(150)
  );

Field	Type	Null	Key	Default	Extra
vehicleID	int	YES		NULL	
model_year	smallint	YES		NULL	
make	varchar(150)	YES		NULL	
model	varchar(100)	YES		NULL	
model_color	varchar(20)	YES		NULL	
vehicleCondition	varchar(150)	YES		NULL	

- ALTER TABLE vehicles
  CHANGE model_year year INT NOT NULL,
  DROP model,
  DROP model_color,
  DROP vehicleCondition;

Field	Type	Null	Key	Default	Extra
vehicleID	int	YES		NULL	
year	int	NO		NULL	
make	varchar(150)	YES		NULL	

alter table

Sample table

```
CREATE TABLE vehicles(
vehicleID INT PRIMARY KEY ,
year INT,
make VARCHAR(100)
);
```

Add new columns to a table

```
ALTER TABLE vehicles
ADD model VARCHAR(100) NOT NULL,
ADD color VARCHAR(50),
ADD note VARCHAR(255);
```

Modify columns

```
ALTER TABLE vehicles
MODIFY year SMALLINT NOT NULL,
MODIFY color VARCHAR(20) NOT NULL,
MODIFY make VARCHAR(150) NOT NULL;
```

Rename columns

```
ALTER TABLE vehicles
CHANGE year model_year SMALLINT NOT NULL,
CHANGE color model_color VARCHAR(20),
CHANGE note vehicleCondition VARCHAR(150);
```

DROP columns

```
ALTER TABLE vehicles
CHANGE model_year year INT NOT NULL,
DROP model,
DROP model_color,
DROP vehicleCondition;
```

drop table

Remember:

- DROP and TRUNCATE are DDL commands, whereas DELETE is a DML command.
- DELETE operations can be rolled back (undone), while DROP and TRUNCATE operations cannot be rolled back (DDL statements are auto committed).
- Dropping a TABLE also drops any TRIGGERS for the table.
- Dropping a TABLE also drops any INDEX for the table.
- Dropping a TABLE will not drops any VIEW for the table.
- If you try to drop a PARENT/MASTER TABLE, it will not get dropped.

drop table

DROP [TEMPORARY] TABLE [IF EXISTS] tbl_name [, tbl_name] ...

Note:

- All table data and the table definition are removed/dropped.
- If it is desired to delete only the records but to leave the table definition for future use, then the ***DELETE*** command should be used instead of ***DROP TABLE***.

- DROP login;
- DROP TABLE users;
- DROP TABLE login, users;

create table using different engines

```
show engines;
set default_storage_engine = memory;
```

create table with memory engine

- **MEMORY** storage engine tables are visible to another client/user.
- Structure is stored and rows will be removed, after re-starting mysql server (MySQL80) from Services.
- Provides in-memory tables, formerly known as HEAP.
- It sores all data in RAM for faster access than storing data on disks.
- Operations involving non-critical data such as session management or caching.

e.g.
```
CREATE TABLE temp(c1 INT,  c2 INT) ENGINE = MEMORY;
```

-
```
INSERT INTO temp VALUES(10, 10);
```
-
```
SELECT * FROM temp;
```

re-start mysql server.

-
```
SELECT * FROM temp;
```

create table with csv engine

```
show engines;
set default_storage_engine = csv;
```

- **CSV** storage engine tables are visible to another client.
- The CSV storage engine stores data in text/csv files using comma-separated values format.
- The storage engine for the table doesn't support nullable (NULL) columns.
- Doesn't support AUTO_INCREMENT columns.
- Doesn't support PRIMARY KEY and UNIQUE KEY constraints.
- CHECK constraint with NOT NULL is allowed.

e.g.
```
CREATE TABLE x(
    ID INT NOT NULL,
    ename VARCHAR(10) NOT NULL,
    job VARCHAR(10) NOT NULL,
    sal INT NOT NULL) ENGINE = CSV;
```

- `INSERT INTO x VALUES(1, 'saleel', 'manager', 3400);`
- `SELECT * FROM x;`

Note:

- ERROR 1194 (HY000): Table 'x' is marked as crashed and should be repaired.
- mysql> REPAIR TABLE x;

create table with blackhole engine

```
show engines;
set default_storage_engine = blackhole;
```

- **BLACKHOLE** tables are visible to another client.
- storage engine acts as a “black hole” that accepts data but throws it away and does not store it.
- Triggers can be written on this type of tables

e.g. CREATE TABLE temp(c1 INT PRIMARY KEY AUTO_INCREMENT, c2 INT UNIQUE, c3 INT NOT NULL, c4 INT CHECK(c4 >= 100)) ENGINE = BLACKHOLE;

- INSERT INTO temp(c2, c3, c4) VALUES(100, 200, 300);
- SELECT * FROM temp;

- DROP TRIGGER IF EXISTS triggername;

```
delimiter $$
CREATE TRIGGER triggername BEFRE INSERT ON temp FOR EACH ROW
begin
   INSERT INTO temp1 VALUES (NEW.c1, NEW.c2);
end $$
delimiter ;
```

create temporary table

- **TEMPORARY** tables are not visible to another client.
- Structure and rows is removed, after exit.

e.g. CREATE TEMPORARY TABLE temp(c1 INT, c2 INT);

- INSERT INTO temp VALUES(10, 10);
- SELECT * FROM temp;
- EXIT

table partitioning

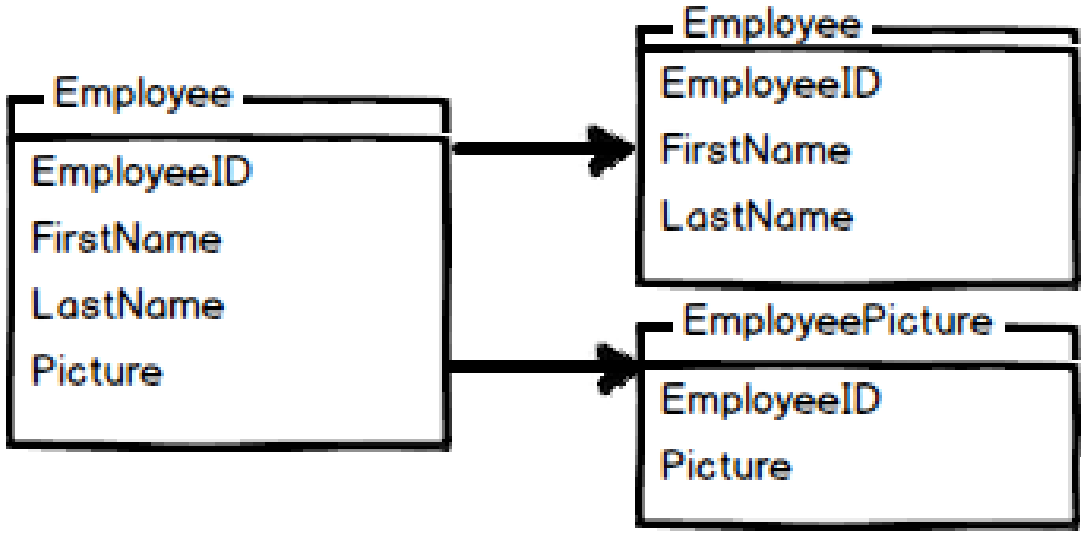
Partitioning separates data into logical units.

table partitioning

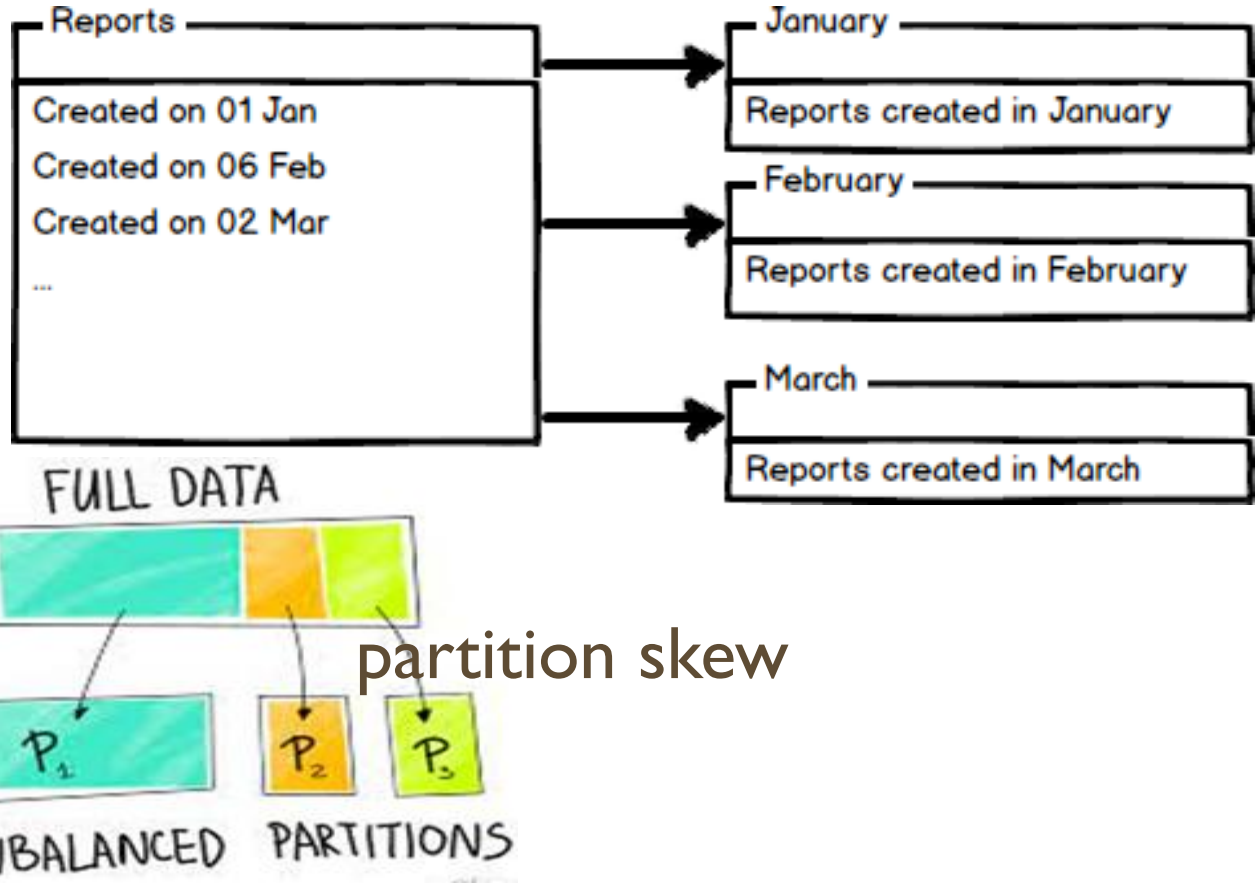
What is a database table partitioning?

Partitioning is the database process where very large tables are divided into multiple smaller parts. By splitting a large table into smaller, individual tables. The main of goal of partitioning is to aid in maintenance of large tables and to reduce the overall response time to read and load data for particular SQL operations.

Vertical Partitioning



Horizontal Partitioning



partition skew

MySQL has mainly six types of partitioning, which are given below:

- RANGE Partitioning
- LIST Partitioning
- COLUMNS Partitioning
- HASH Partitioning
- KEY Partitioning
- Subpartitioning

table partitioning

Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Vertical Partitions

VP1

CUSTOMER ID	FIRST NAME	LAST NAME
1	TAEKO	OHNUKI
2	O.V.	WRIGHT
3	SELDA	BAĞCAN
4	JIM	PEPPER

VP2

CUSTOMER ID	FAVORITE COLOR
1	BLUE
2	GREEN
3	PURPLE
4	AUBERGINE

Horizontal Partitions

HP1

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN

HP2

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

partitioning by range / list

RANGE Partitioning

```
PARTITION BY RANGE (COLUMNS)
  (
    PARTITION part_name1 VALUES LESS THAN (int_value),
    PARTITION part_name2 VALUES LESS THAN (int_value),
    PARTITION part_name3 VALUES LESS THAN MAXVALUE
  )
```

LIST Partitioning

```
PARTITION BY LIST (COLUMNS)
  (
     PARTITION part_name1 VALUES IN (int_value_list),
     PARTITION part_name2 VALUES IN (int_value_list),
     PARTITION part_name3 VALUES IN (int_value_list)
  )
```

RANGE Partitioning

```
e.g. CREATE TABLE employee (
        empno  INT,
        ename  VARCHAR(10),
        salary INT
     )
     PARTITION BY RANGE (salary) (
       PARTITION p0 VALUES LESS THAN (2000),
       PARTITION p1 VALUES LESS THAN (4000),
       PARTITION p2 VALUES LESS THAN (6000),
       PARTITION p3 VALUES LESS THAN MAXVALUE
     );
```

- INSERT INTO employee PARTITION(p0) VALUES(1, 'saleel', 1500);
- SELECT * FROM employee PARTITION(p0);
- UPDATE employee PARTITION(p0) set salary = 1500;
- UPDATE employee PARTITION(p0) set salary = 3000 WHERE empno = 1; // Invalid statement
- DELETE FROM employee PARTITION(p0);

RANGE Partitioning

```
e.g. CREATE TABLE bookings (
       booking_id INT,
       guest_name VARCHAR(100),
       checkin_date DATE
     )
     PARTITION BY RANGE(MONTH(checkin_date)) (
       PARTITION January VALUES LESS THAN (2),
       PARTITION February VALUES LESS THAN (3),
       PARTITION March VALUES LESS THAN (4),
       PARTITION April VALUES LESS THAN (5),
       PARTITION May VALUES LESS THAN (6),
       PARTITION June VALUES LESS THAN (7),
       PARTITION July VALUES LESS THAN (8),
       PARTITION August VALUES LESS THAN (9),
       PARTITION September VALUES LESS THAN (10),
       PARTITION October VALUES LESS THAN (11),
       PARTITION November VALUES LESS THAN (12),
       PARTITION December VALUES LESS THAN (13)
     );
```

LIST Partitioning

```
e.g. CREATE TABLE item (
        itemID  INT,
        itemDesc  VARCHAR(10),
        storeID INT
     )
    PARTITION BY LIST(storeID) (
       PARTITION p0 VALUES IN(1, 3, 5, 7),
       PARTITION p1 VALUES IN(2, 4, 6, 8),
       PARTITION p2 VALUES IN(10, 12, 14, 16),
       PARTITION p3 VALUES IN(11, 13, 15, 17)
    );
```

Warehouse	storeID
AC Warehouse	1, 3, 5, 7
National	2, 4, 6, 8
Global	10, 12, 14, 16
Migrant System	11, 13, 15, 17



```
CREATE TABLE customers (
   id INT,
   name VARCHAR(50),
   region VARCHAR(20)
 )
 PARTITION BY LIST COLUMNS(region) (
   PARTITION east VALUES IN ('East', 'Northeast'),
   PARTITION west VALUES IN ('West', 'Southwest'),
   PARTITION central VALUES IN ('Central'),
   PARTITION other VALUES IN ('North', 'South')
);
```

alter / drop partitioning by range / list

Alter Partitioning

RANGE Partitioning

```
ALTER TABLE a ADD PARTITION (PARTITION p3 VALUES LESS THAN(130));
```

- MAXVALUE can only be used in last partition definition

LIST Partitioning

```
ALTER TABLE a ADD PARTITION (PARTITION p3 VALUES IN (10, 11));
```

DROP Partitioning

```
ALTER TABLE a ADD PARTITION p3;
```

create temporary table

Note:

- it is possible to create, alter, drop, and write (Insert, Update, and Delete rows) to TEMPORARY tables.
- You can use all constraints (PRIMARY KEY, UNIQUE, DEFAULT, etc.).
- You can create generated column in temporary table.

temporary table

Remember:

- You can use the *TEMPORARY* keyword when creating a table.
- A *TEMPORARY* table is visible only to the current session, and is dropped automatically when the session is closed.
- Use *TEMPORARY* table with the same name as the original can be useful when you want to try some statements that modify the contents of the table, without changing the original table.
- The permanent (original) table becomes hidden (inaccessible) to the client who creates the *TEMPORARY* table with same name as the original.
- If you issue a DROP TABLE statement, the *TEMPORARY* table is removed and the original table reappears, it is possible, only when then original *tbl_name* and temporary *tbl_name* are same.
- The original table also reappears if you rename the *TEMPORARY* table.

  e.g. ALTER TABLE **dept** RENAME TO d;

  Temporary table_name

temporary table

e.g.

```
CREATE TEMPORARY TABLE student (
    ID INT PRIMARY KEY,
    namefirst VARCHAR(45),
    namelast VARCHAR(45),
    DOB DATE,
    emailID VARCHAR(128)
) ;

CREATE TEMPORARY TABLE temp (
    ID INT PRIMARY KEY,
    firstName VARCHAR(45),
    phone INT,
    city VARCHAR(10) DEFAULT 'PUNE',
    salary INT,
    comm INT,
    total INT GENERATED ALWAYS AS(salary + comm) VIRTUAL
) ;
```

create temporary table ... like

Use CREATE TABLE ... LIKE to create an empty table based on the definition of another table.

```
CREATE TEMPORARY TABLE [IF NOT EXISTS] new_tbl LIKE orig_tbl;
```

- `CREATE TEMPORARY TABLE tempEmployee LIKE employee;`

Remember:

- LIKE works only for base tables, not for VIEWS.
- You can use the TEMPORARY keyword when creating a table. A TEMPORARY table is visible only to the current session, and is dropped automatically when the session is closed.
- Use TEMPORARY table with the same name as the original can be useful when you want to try some statements that modify the contents of the table, without changing the original table.

- `CREATE TEMPORARY TABLE new_tbl SELECT * FROM orig_tbl LIMIT 0;`

Do not use the * operator in your SELECT statements. Instead, use column names. Reason is that in MySQL Server scans for all column names and replaces the * with all the column names of the table(s) in the SELECT statement. Providing column names avoids this search-and-replace, and enhances performance.

continue with SELECT statement…

```
SELECT what_to_select
FROM which_table
WHERE conditions_to_satisfy;
```

The asterisk symbol " * " can be used in the SELECT clause to denote "all attributes."

Capabilities of SELECT Statement

1. SELECTION
2. PROJECTION
3. JOINING

Capabilities of SELECT Statement

➢ SELECTION

Selection capability in SQL is to choose the rows in a table that you want to return by a query.

R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	10
2	Janhavi	Sales	1994-12-20	20
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	10
5	Ketan	Sales	1994-01-01	30

Capabilities of SELECT Statement

➢ PROJECTION

Projection capability in SQL to choose the columns in a table that you want to return by your query.

R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	10
2	Janhavi	Sales	1994-12-20	20
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	10
5	Ketan	Sales	1994-01-01	30

Capabilities of SELECT Statement

➢ JOINING

Join capability in SQL to bring together data that is stored in different tables by creating a link between them.

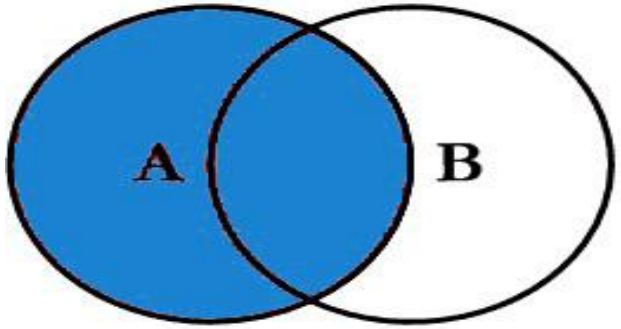
R

EMPNO	ENAME	JOB	HIREDATE	DEPTNO
1	Saleel	Manager	1995-01-01	20
2	Janhavi	Sales	1994-12-20	10
3	Snehal	Manager	1997-05-21	10
4	Rahul	Account	1997-07-30	20
5	Ketan	Sales	1994-01-01	30

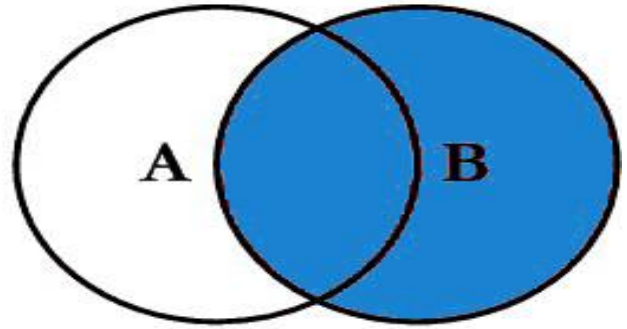
S

DEPTNO	DNAME	LOC
10	HRD	PUNE
20	SALES	BARODA
40	PURCHASE	SURAT

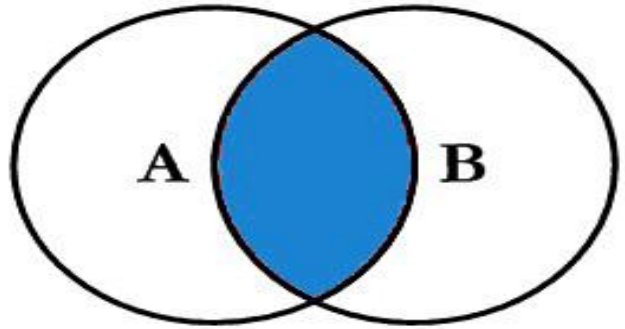
SQL JOINS



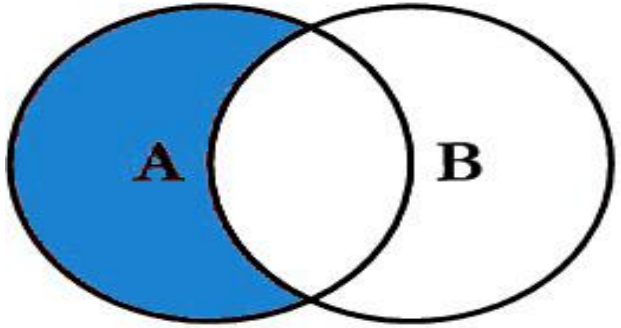

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



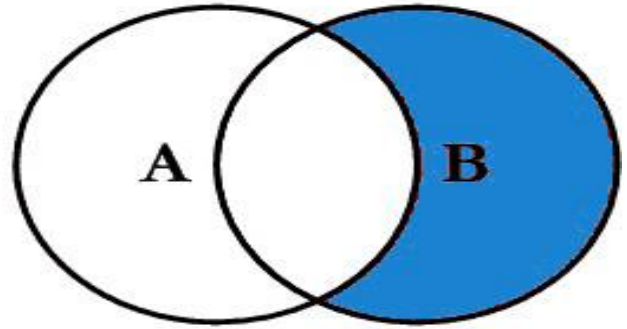

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



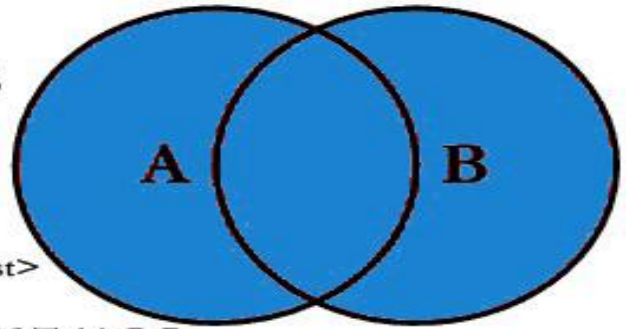

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



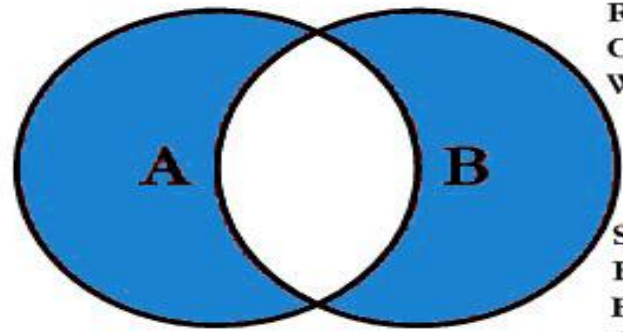

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

select statement

SELECTION Process

SELECT * FROM <table_references>

selection-list | field-list | column-list

When you write a SELECT query, the list of columns or expressions you mention after the SELECT keyword is called the SELECT-LIST (also referred to as FIELD-LIST).

Remember:

- Here, " * " is known as metacharacter (all columns)

PROJECTION Process

SELECT column-list FROM <table_references>

selection-list | field-list | column-list

Remember:

- Position of columns in SELECT statement will determine the position of columns in the output (as per user requirements)

syntax

select statement

modifiers

SELECT [ALL / DISTINCT / DISTINCTROW] identifier.* / identifier.A_1 [[as] *alias_name*], identifier.A_2 [[as] *alias_name*], identifier.A_3 [[as] *alias_name*], expression1 [[as] *alias_name*], expression2 [[as] *alias_name*] . . .

- [FROM <identifier.r_1> [as] *alias_name*], <identifier.r_2> [as] *alias_name*], . . .]
- [WHERE < *where_condition1* > { and | or } < *where_condition2* > . . .]
- [GROUP BY < { *col_name* | *expr* | *position* }, . . . [WITH ROLLUP] >]
- [HAVING < *having_condition1* > { and | or } < *having_condition2* > . . .]
- [ORDER BY < { *col_name* | *expr* | *position* } [ASC | DESC], . . . >]
- [LIMIT < { [*offset*,] *row_count* | *row_count* OFFSET *offset* } >]
- [FOR { UPDATE }]
- [{ INTO OUTFILE '*file_name*' | INTO DUMPFILE '*file_name*' | INTO *var_name* [, *var_name*], . . . }]

found_rows() (returns the number of rows)

Remember:

- **ALL** (**modifier is default**) specifies that all matching rows should be returned, including duplicates.
- **DISTINCT** (**modifier**) specifies removal of duplicate rows from the result set.
- **DISTINCTROW** (**modifier**) is a synonym for **DISTINCT**.
- It is an error to specify both modifiers.
- Whenever you use **DISTINCT**, sorting takes place in server.

- SELECT 'HELLO' ' WORLD';
- SELECT 'HELLO' AS 'WORLD';
- SELECT ename `EmployeeName` FROM emp;
- SELECT ename AS `EmployeeName` FROM emp;

column - alias

A programmer can use an alias to temporarily assign another name to a **column** or **table** for the duration of a ***SELECT*** query**.**

In the selection-list, a quoted column alias can be specified using identifier **(`)** or string quote **(' or ")** characters.

Note:

- Assigning an alias_name does not actually rename the column or table.
- You cannot use alias in an expression.

select statement - alias

SELECT A_1 [[AS] *alias_name*], A_2 [[AS] *alias_name*], . . ., A_N FROM ***r*** [[AS*] alias_name*]

column-name as new-name

table-name as new-name

Remember:

- A select_expr can be given an alias using **AS alias_name**. The alias is used as the expression's column name and can be used in **GROUP BY, HAVING,** or **ORDER BY** clauses.
- The **AS** keyword is optional when aliasing a select_expr with an identifier.
- Standard SQL **disallows** references to column aliases in a ***WHERE*** clause.
- A table reference can be aliased using **tbl_name alias_name** or **tbl_name AS alias_name**
- If the column alias contains spaces, **put it in quotes**.
- Alias name is **max 256 characters**.

- SELECT empno AS *EmployeeID*, ename *EmployeeName* FROM emp;
- SELECT ID AS *'Employee ID'*, ename *"Employee Name"* FROM emp;
- SELECT * FROM emp *employee*;

comparison functions and operator

Comparison operations result in a value of 1 **(TRUE),** 0 **(FALSE)**, or **NULL**.

assignment _operator

= (assignment), :=

comparison functions and operator

1. ***arithmetic_operators:***
   * | / | DIV | % |MOD | - | +
2. ***comparison_operator:***
   = | <=> | >= | > | <= | < | <> | !=
3. ***boolean_ predicate:***
   IS [NOT] *NULL* | IS [BOOLEAN]
   | *expr* <=> *null*
4. ***predicate*:**
   expr [NOT] LIKE *expr* [ESCAPE *char*]
   | *expr* [NOT] IN (*expr1, expr2, . . .*)
   | *expr* [NOT] IN (*subquery*)
   | *expr* [NOT] BETWEEN *expr1* AND *expr2*
5. ***logical_operators***
   { AND | && } | { OR | || }
6. ***assignment _operator***
   = (assignment), :=

operand meaning: the quantity on which an operation is to be done.

e.g.

1. *operand1 * operand2*
2. *operand1 = operand2*
3. *operand* IS [NOT] *NULL*
4. *operand* [NOT] LIKE 'pattern'
5. *expr* AND *expr*
6. *Operand := 1001*

- SELECT 23 DIV 6 ; #3
- SELECT 23 / 6 ; #3 .8333

Note:

- NOT has higher precedence than AND **and** AND has higher precedence than OR

- `WHERE col * 4 < 16`
- `WHERE col < 16 / 4`

- `SELECT CONCAT(1, "saleel");`

column - expressions

"Strings are automatically converted to numbers (this behavior is known as **implicit type conversion (type coercion)** in MySQL.)."

- If a string starts with a number, MySQL extracts the number and uses it.
- If a string does not start with a number, MySQL converts it to 0.

select statement - expressions

Column EXPRESSIONS

SELECT A_1, A_2, A_3, A_4, expressions, . . . FROM *r*

- SELECT 1001 + 1;
- SELECT 1001 + '1';
- SELECT '1' + '1' ;
- SELECT '1' + 'a1';
- SELECT '1' + '1a';
- SELECT 'a1' + 1;
- SELECT '1a' + 1;
- SELECT 1 + -1;
- SELECT 1 + -2;
- SELECT -1 + -1;
- SELECT -1 - 1;
- SELECT -1 - -1;
- SELECT 123 * 1;
- SELECT -123 * 1;
- SELECT 123 * -1 ;
- SELECT -123 * -1;
- SELECT 2 * 0;
- SELECT 2435 / 1;
- SELECT 2 / 0;
- SELECT '2435Saleel' / 1;
- SELECT NULL <=> NULL;
- SELECT '20' > 15;
- SELECT 'abc' > 0;
- SELECT 'abc' = 0;
- SELECT '123abc' = 123;
- SELECT 'abc123' = 0;
- SELECT sal, sal + 1000 AS *'New Salary'* FROM emp;
- SELECT sal, comm, sal + comm FROM emp;
- SELECT sal, comm, sal + IFNULL(comm, 0) FROM emp;
- SELECT ename, ename = ename FROM emp;
- SELECT ename, ename = 'smith' FROM emp;
- SELECT c1, c1 / 1 *R1* FROM numberString;

Note:

If any expression evaluated with NULL, returns NULL.

- SELECT 2 + NULL ;
- SELECT 2 - NULL ;
- SELECT 2 * NULL ;
- SELECT 2 / NULL ;
- SELECT 2 % NULL ;

Note:

In SQL, the same name can be used for two (or more) attributes as long as the attributes are in different relations. If this is the case, and a multi-table query refers to two or more attributes with the same name, we must ***qualify*** the attribute name with the relation name to prevent ambiguity. This is done by prefixing the relation name to the attribute name and separating the two by a period (**.**).

identifiers

Certain objects within MySQL, including database, table, index, column, alias, view, stored procedure, stored functions, triggers, partition, tablespace, and other object names are known as **identifiers**.

identifiers

The maximum length for each type of identifiers like (Database, Table, Column, Index, Constraint, View, Stored Program, Compound Statement Label, User-Defined Variable, Tablespace) is 64 characters, whereas for Alias is 256 characters.

- You can refer to a table within the default database as
  1. tbl_name
  2. db_name.tbl_name.

- You can refer to a column as
  1. col_name
  2. tbl_name.col_name
  3. db_name.tbl_name.col_name.

Note:

- You need not specify a ***tbl_name*** or ***db_name.tbl_name*** prefix for a column reference unless the reference would be ambiguous.
- The identifier quote character is the backtick (`)

- NULL value does not occupy space in memory.
- NULL value is independent of data type.
- NULL can be written in any lettercase.
- NULL represents a missing or unknown value.
- It is not a number (0) or an empty string ("")
- A missing / unknown value cannot be compared as equal or not-equal
- SQL uses three-valued logic: TRUE, FALSE and UNKNOWN.

null-to-null comparisons

- SELECT NULL = NULL, NULL IS NULL; // Output will be NULL and 1
- SELECT NULL = NULL, NULL IS NOT NULL; // Output will be NULL and 0

- SELECT FALSE AND NULL;
- SELECT FALSE OR NULL;
- SELECT TRUE AND NULL;
- SELECT TRUE OR NULL;

- SELECT NULL = NULL;
- SELECT NULL <> NULL;
- SELECT NULL is NULL;
- SELECT NULL is NOT NULL;

control flow functions

control flow functions - ifnull

IFNULL function

MySQL IFNULL() takes two expressions, if the first expression is not NULL, it returns the first expression. Otherwise, it returns the second expression, **it returns either numeric or string value.**

IFNULL(*expression1*, *expression2*)

- SELECT IFNULL (1, 2) AS *R1*;
- SELECT IFNULL (NULL, 2) AS *R1*;
- SELECT IFNULL (1/0, 2) AS *R1*;
- SELECT IFNULL (1/0, 'Yes') AS *R1*;
- SELECT comm, IFNULL(comm + comm*.25, 1000) FROM emp;

control flow functions - if

IF function

If **expr1 is TRUE or expr1 <> 0 or expr1 <> NULL**, then IF() returns expr2, otherwise it returns expr3, **it returns either numeric or string value.**

IF(*condition, value_if_true, value_if_false*)

A year is a leap year if:
- Divisible by **400** → leap year
- Divisible by **4** but **not by 100** → leap year
- Otherwise → **not a leap year**

- SELECT IF(1 > 2, 2, 3) as *R1*;
- SELECT sal, IF(sal = 3000, 'Ok', 'Not Bad') *R1* FROM emp;
- SELECT ename, sal, IF(sal = 3000 AND ename = 'FORD', 'Y', 'N') *R1* FROM emp;
- SELECT ename, sal, comm, IF(comm IS NULL && ename = 'FORD', 'Y', 'N') *R1* FROM emp;
- SELECT deptno, IF(deptno = 10, 'Sales', IF(deptno = 20 , 'Purchase' ,'N/A')) *R1* FROM emp;
- SELECT productid, productname, unitprice, unitsinstock, reorderlevel, IF(unitsinstock < reorderlevel, 'Stock is less', 'Good Stock') as 'Stock Report' FROM products;
- SELECT hiredate, IF((YEAR(hiredate) % 4 = 0 AND YEAR(hiredate) % 100 <> 0) OR YEAR(hiredate) % 400 = 0 ,'Leap Year', 'Not A Leap Year') FROM emp;

control flow functions - nullif

NULLIF function

Returns **NULL** if expr1 = expr2 is true, otherwise returns expr1.

NULLIF(*expr1*, *expr2*)

- SELECT NULLIF(1, 1) as *R1*;
- SELECT NULLIF(1, 2) as *R1*;

control flow functions - case

CASE function

Returns the result for the first condition that is true. If there was no matching result value, the result after ELSE is returned, or NULL if there is no ELSE part.

```
CASE value WHEN [compare_value] THEN result [WHEN [compare_value] THEN result . . .] [ELSE result] END
```

- `SELECT deptno, CASE deptno WHEN 10 THEN 'Accounts' WHEN 20 THEN 'Sales' ELSE 'N/A' END R1 FROM emp;`
- `SELECT deptno, CASE deptno WHEN 10 THEN 'Accounts' ELSE 'N/A' END CASE FROM emp; # error`
- `SELECT custId, type, amount, CASE type WHEN 'd' THEN amount WHEN 'c' THEN amount * -1 END amount FROM transactions;`

control flow functions - case

CASE function

Returns the result for the first condition that is true. If there was no matching result value, the result after ELSE is returned, or NULL if there is no ELSE part.

```
CASE WHEN [condition] THEN result [WHEN [condition] THEN result . . .] [ELSE result] END
```

- `SELECT deptno, CASE WHEN deptno = 10 THEN 'Sales' WHEN deptno = 20 THEN 'Purchase' ELSE 'N/A' END R1 FROM emp;`
- `SELECT companyName, CASE WHEN country IN ('USA', 'Canada') THEN 'North America' WHEN country = 'Brazil' THEN 'South America' WHEN country IN ('Japan', 'Singapore') THEN 'Asia' WHEN country = 'Australia' THEN 'Australia' ELSE 'Europe' END as Continent FROM suppliers ORDER BY companyName;`
- `SELECT hiredate, CASE WHEN (YEAR(hiredate) % 4 = 0 AND YEAR(hiredate) % 100 <> 0) OR YEAR(hiredate) % 400 = 0 THEN 'LEAP YEAR' END R1 FROM emp;`

control flow functions - case

CASE function

Returns the result for the first condition that is true. If there was no matching result value, the result after ELSE is returned, or NULL if there is no ELSE part.

CASE WHEN [*condition*] THEN result [WHEN [*condition*] THEN result . . .] [ELSE *result*] END

```
* Count(custID)
  ORDER BY CASE orderCount
  WHEN 1 THEN 'One-time Customer'
  WHEN 2 THEN 'Repeated Customer'
  WHEN 3 THEN 'Frequent Customer'
  ELSE 'Loyal Customer' END customerType
```

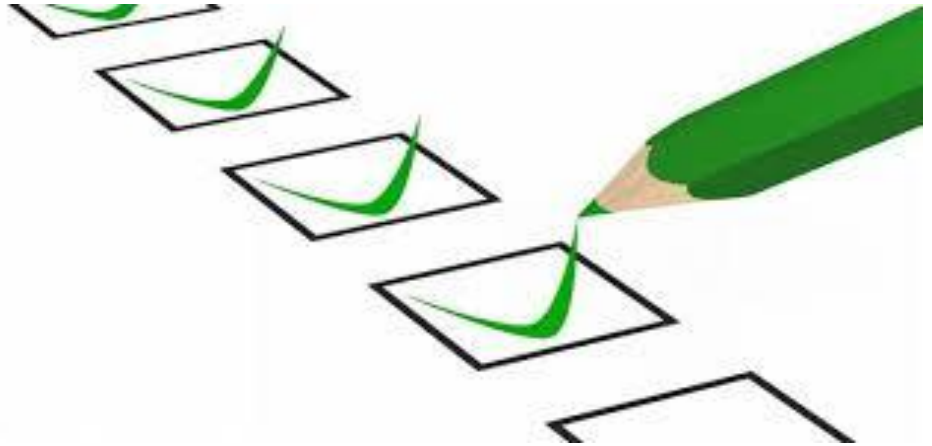
```
* ORDER BY CASE
  WHEN filter = 'Debit' THEN 1
  WHEN filter = 'Credit' THEN 2
  WHEN filter = 'Total' THEN 3
  END transactionType;
```

```
* ORDER BY FIELD(status, 'In Process',
  'On Hold', 'Cancelled', 'Resolved',
  'Disputed', 'Shipped');
```

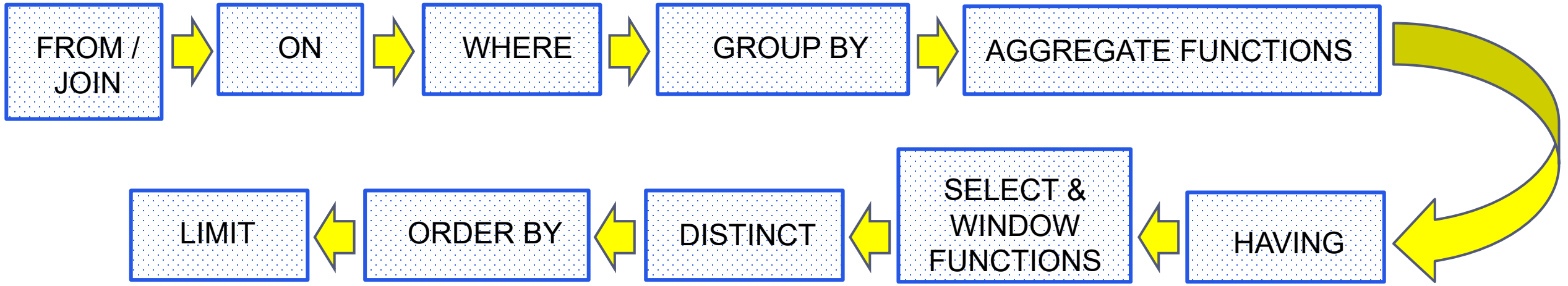
```
* ORDER BY CASE status
  WHEN 'active' THEN 1
  WHEN 'approved' THEN 2
  WHEN 'rejected' THEN 3
  WHEN 'submitted' THEN 4
  ELSE 5 END statusType
```

sequence of clauses





select statement... execution



- DATEDIFF(CURDATE(), hiredate) / 365.25

datetime functions

sysdate(), now(), curdate(), curtime()

In MySQL, the **NOW()** function returns a default value for a **DATETIME**.
MySQL inserts the current **date and time** into the column whose default value is NOW().

In MySQL, the **CURDATE()** returns the current date in 'YYYY-MM-DD'. **CURRENT_DATE()** and **CURRENT_DATE** are the **synonym of CURDATE().**

In MySQL, the **CURTIME()** returns the value of current time in 'HH:MM:SS'. **CURRENT_TIME()** and **CURRENT_TIME** are the **synonym of CURTIME().**

+ or - operator

Date arithmetic also can be performed using INTERVAL together with the + or - operator

date + INTERVAL *expr* unit + INTERVAL *expr* unit + INTERVAL *expr* unit + . . .
date - INTERVAL *expr* unit - INTERVAL *expr* unit - INTERVAL *expr* unit - . . .

- SELECT NOW(), NOW() + INTERVAL 1 DAY;
- SELECT NOW(), NOW() + INTERVAL '1-3' YEAR_MONTH;

unit Value	expr	unit Value	expr
SECOND	SECONDS	DAY_HOUR	'DAYS HOURS' e.g. '1 1'
MINUTE	MINUTES	DAY_MINUTE	'DAYS HOURS:MINUTES' e.g. '1 3:34'
HOUR	HOURS	DAY_SECOND	'DAYS HOURS:MINUTES:SECONDS'
DAY	DAYS	HOUR_MINUTE	'HOURS:MINUTES' e.g. '3:34'
WEEK	WEEKS	HOUR_SECOND	'HOURS:MINUTES:SECONDS'
MONTH	MONTHS	MINUTE_SECOND	'MINUTES:SECONDS' e.g. '27:34'
QUARTER	QUARTERS	YEAR_MONTH	'YEARS-MONTHS' e.g. '1-3'
YEAR	YEARS		

extract

The EXTRACT() function is used to return a single part of a date/time, such as year, month, day, hour, minute, etc.

EXTRACT(*unit* FROM *date*)

Unit Value				
MICROSECOND	SECOND	MINUTE	HOUR	DAY
WEEK	MONTH	QUARTER	YEAR	
MINUTE_SECOND	HOUR_SECOND	DAY_SECOND	DAY_HOUR	
HOUR_MINUTE	DAY_MINUTE	YEAR_MONTH		

- SELECT EXTRACT(MONTH FROM NOW());
- SELECT EXTRACT(YEAR_MONTH FROM NOW()) ;

Note:

- There must no space between extract function and ().

e.g.

SELECT EXTRACT (MONTH FROM NOW()); # error

datetime functions

Syntax	Result
DAY(*date*)	DAY() is a **synonym for DAYOFMONTH().**
DAYNAME(*date*)	Returns the name of the weekday for date.
DAYOFMONTH(*date*)	Returns the day of the month for date, in the range 1 to 31
DAYOFWEEK(*date*)	Returns the weekday index for date (1 = Sunday, 2 = Monday, …, 7 = Saturday).
DAYOFYEAR(*date*)	Returns the day of the year for date, in the range 1 to 366
LAST_DAY(*date*)	Takes a date or datetime value and returns the corresponding value for the last day of the month. Returns NULL if the argument is invalid.
MONTH(*date*)	Returns the month for date, in the range 1 to 12 for January to December
MONTHNAME(*date*)	Returns the full name of the month for date.
YEAR(*date*)	Returns the year in 4 digit

- SELECT DAYOFWEEK(NOW()), WEEKDAY(NOW());
- SELECT DAYOFWEEK(ADDDATE(NOW(), INTERVAL 1 DAY)), WEEKDAY(ADDDATE(NOW(), INTERVAL 1 DAY));

datetime functions

Syntax	Result
WEEKDAY(*date*)	Returns the weekday index for date (0 = Monday, 1 = Tuesday, … 6 = Sunday).
WEEKOFYEAR(*date*)	Returns the calendar week of the date as a number in the range from 1 to 53.
QUARTER(*date*)	Returns the quarter of the year for date, in the range 1 to 4.
HOUR(*time*)	Returns the hour for time. The range of the return value is 0 to 23 for time-of-day values.
MINUTE(*time*)	Returns the minute for time, in the range 0 to 59.
SECOND(*time*)	Returns the second for time, in the range 0 to 59.
DATEDIFF(*expr1, expr2*)	Returns the number of days between two dates or datetimes.
STR_TO_DATE(*str, format*)	Convert a string to a date.

- `SELECT NOW(), NOW() + INTERVAL 1 DAY, WEEKDAY(NOW() + INTERVAL 1 DAY);`
- `SELECT * FROM emp WHERE DAY(hiredate) = 17;`
- `SELECT YEAR(hiredate), ( YEAR(hiredate) % 4 = 0 AND YEAR(hiredate) % 100 != 0 ) OR YEAR(hiredate) % 400 = 0 R1 FROM emp ;`
- `SELECT STR_TO_DATE('24/05/2022', '%d/%m/%Y');`

datetime formats

datetime formats

Formats	Description
%a	Abbreviated weekday name (Sun-Sat)
%b	Abbreviated month name (Jan-Dec)
%c	Month, numeric (1-12)
%D	Day of month with English suffix (0th, 1st, 2nd, 3rd, □)
%d	Day of month, numeric (01-31)
%e	Day of month, numeric (1-31)
%f	Microseconds (000000-999999)
%H	Hour (00-23)
%h	Hour (01-12)

- SELECT DATE_FORMAT(NOW(), '%a');
- SELECT DATE_FORMAT(CURDATE(), '%Y-%m-01') AS *"First Day"*, LAST_DAY(CURDATE()) AS *"Last Day"*;
- SELECT hiredate, DATE_FORMAT(hiredate, '%Y-%m-01') AS *"First Day"*, LAST_DAY(hiredate) AS *"Last Day"* FROM emp;

datetime formats

Formats	Description
%I	Hour (01-12)
%i	Minutes, numeric (00-59)
%j	Day of year (001-366)
%k	Hour (0-23)
%l	Hour (1-12)
%M	Month name (January-December)
%m	Month, numeric (01-12)
%p	AM or PM
%r	Time, 12-hour (hh:mm:ss followed by AM or PM)
%S	Seconds (00-59)
%s	Seconds (00-59)

- `SELECT DATE_FORMAT(NOW(), '%j');`

datetime formats

Formats	Description
%T	Time, 24-hour (hh:mm:ss)
%U	Week (00-53) where Sunday is the first day of week
%u	Week (00-53) where Monday is the first day of week
%V	Week (01-53) where Sunday is the first day of week, used with %X
%v	Week (01-53) where Monday is the first day of week, used with %x
%W	Weekday name (Sunday-Saturday)
%w	Day of the week (0=Sunday, 6=Saturday)
%X	Year for the week where Sunday is the first day of week, four digits, used with %V
%x	Year for the week where Monday is the first day of week, four digits, used with %v
%Y	Year, numeric, four digits
%y	Year, numeric, two digits

- `SELECT DATE_FORMAT(NOW(), '%Y');`

string functions

string functions

Syntax	Result
ASCII(*str*)	Returns the numeric value of the leftmost character of the string str. Returns 0 if str is the empty string. Returns NULL if str is NULL. e.g. • SELECT ASCII(ename) FROM emp;
CHAR(*N*, , . . .)	CHAR() interprets each argument N as an integer and returns a string consisting of the characters given by the code values of those integers. **NULL values are skipped**. e.g. • SELECT CHAR(65, 66, 67); / SELECT CAST(CHAR(65 66, 67) AS CHAR);
CONCAT(*str1, str2*, . . .)	Returns the string that results from concatenating the arguments. CONCAT() **returns NULL if any argument is NULL**. e.g. • SELECT CONCAT('Mr. ' , ename) FROM emp; • SELECT CONCAT('My', NULL, 'SQL'); #output will be NULL
ELT(*N, str1, str2, str3,* . . .)	ELT() returns the Nth element of the list of strings: str1 if N = 1, str2 if N = 2, and so on. Returns NULL if N is less than 1 or greater than the number of arguments. • SELECT hiredate, ELT(MONTH(hiredate),'Winter', 'Winter', 'Spring', 'Spring', 'Spring', 'Summer', 'Summer', 'Summer', 'Autumn', 'Autumn', 'Autumn', 'Winter') R1 FROM emp;

- SELECT Name, rnk, ELT(rnk, 'A+', 'A', 'B', 'C', 'D') *grade* FROM student;
- SELECT bankName, accountNumber, riskStatus, CASE WHEN riskStatus = 0 THEN 'No risk' ELSE ELT(riskStatus, 'Low', 'Moderate', 'High', 'Critical') END *riskStatus* FROM bankLoan;

string functions

Syntax	Result
STRCMP(*expr1, expr2*)	STRCMP() returns 0 if the strings are the same, -1 if the first argument is smaller than the second according to the current sort order, and 1 otherwise.
LCASE(*str*)	Returns lower case string. LCASE() **is a synonym for** LOWER().
UCASE(*str*)	Returns upper case string. UCASE() **is a synonym for** UPPER().
LENGTH(*str*)	Returns the length of the string.
LPAD(*str, len, padstr*)	Returns the string str, left-padded with the string padstr to a length of len characters.
RPAD(*str, len, padstr*)	Returns the string str, right-padded with the string padstr to a length of len characters.
REPEAT(*str, count*)	Returns a string consisting of the string str repeated count times. If count is less than 1, returns an empty string. Returns NULL if str or count are NULL.

- SELECT UCASE(ename) FROM emp;
- SELECT LPAD(id, 7, '#'), bookName, Type, Cost FROM books;
- SELECT LPAD(prodid, 9, 'x'), Descrip, Price FROM product;
- SELECT CONCAT('TKT', LPAD(id, 7, 0)) *'Ticket ID'* FROM hall_ticket;
- SELECT COUNT(*) *R1* FROM emp WHERE BINARY ename = BINARY UPPER(ename);
- SELECT CASE WHEN status = 'In Process' THEN LPAD(ordid, LENGTH(ordid) + 3, '(I)') WHEN status = 'Delivered' THEN LPAD(ordid, LENGTH(ordid) + 3, '(D)') ELSE 'TODO' END ordid, orderDate, custID, Status, Total FROM ord;

string functions

Syntax	Result
LEFT(*str, len*)	Returns the leftmost len characters from the string str, or NULL if any argument is NULL.
RIGHT(*str, len*)	Returns the rightmost len characters from the string str, or NULL if any argument is NULL.
LTRIM(*str*)	Returns the string str with leading space characters removed.
RTRIM(*str*)	Returns the string str with trailing space characters removed.
TRIM(*str*)	Returns the string str with leading and trailing space characters removed.
BINARY *value*	Convert a value to a binary string.
INSTR(*str, substr*)	• *str,* The string that will be searched • *substr,* The string to search for in string1. If string2 is not found, this function returns 0
FIND_IN_SET(*str*, *string_list*)	• *str,* The string to search for • *string_list,* The list of string values to be searched (separated by commas)

FIND_IN_SET:- This function of MySQL is used to find the position of a string within a comma-separated list of strings.

- If *string* is not found in *string_list*, this function returns 0
- If *string* or *string_list* is NULL, this function returns NULL
- If *string_list* is an empty string (""), this function returns 0

- SELECT ename, BINARY ename FROM emp;
- SELECT CONCAT(UCASE(LEFT(ename, 1)), LCASE(SUBSTRING(ename, 2))) *"Title Case"* FROM emp;

SUBSTRING_INDEX(*str*, *delimiter*, *count*) **// count can be either +ve or -ve**

string functions

Syntax	Result
REPLACE(*str*, *from_str*, *to_str*)	Returns the string str with all occurrences of the string from_str replaced by the string to_str. REPLACE() performs a case-sensitive match when searching for from_str. e.g. • SELECT REPLACE('Hello', 'l', 'x');
REVERSE(*str*)	Returns the string str with the order of the characters reversed.
SUBSTR(*str*, *pos*, *len*)	**SUBSTR() is a synonym for SUBSTRING().** e.g. • SELECT SUBSTR ('This is the test by IWAY', 6); • SELECT SUBSTR ('This is the test by IWAY', -4, 4);
MID(*str*, *pos*, *len*)	MID function **is a synonym for** SUBSTRING**.**

- SELECT ename, job, IF(ISNULL(phone), '**********', RPAD(LEFT(phone, 4), 10, '*')) FROM emp;
- SELECT ename, job, phone, IF(ISNULL(phone), REPEAT('*',10), RPAD(LEFT(phone, 4), 10, '*')) FROM emp;
- SELECT `user name`, IF(LENGTH(SUBSTR(`user name`, INSTR(`user name`," "))) = 0,"Weak User", `user name`) *R1* FROM emp;
- UPDATE emp SET job = REPLACE(job, job, LOWER(job));

string functions - examples

- SELECT sal, REPEAT('$', sal/100) FROM emp;
- SELECT emailid, REPEAT('*', LENGTH(emailid)) FROM emp;
- SELECT pwd, REPEAT('*', LENGTH(pwd)) *password* FROM emp;
- SELECT c1, CONCAT(REPEAT('0', 10 - LENGTH(c1)) , c1) FROM leading_zeroes;
- SELECT ename, job, IF(ISNULL(phone), '**********', RPAD(LEFT(phone, 4), 10, '*')) FROM emp;
- SELECT 9850884228, LPAD(SUBSTR(9850884228, -4),10,'*') *R1*;
- SELECT `user name`, IF(LENGTH(SUBSTR(`user name`, INSTR(`user name`," "))) = 0,"Weak User", `user name`) *R1* FROM emp;
- SELECT LENGTH('saleel') - LENGTH(REPLACE('saleel', 'e', ""));
- SELECT empno, datePresent, LENGTH(datePresent) - LENGTH(REPLACE(datePresent, ",", "")) + 1 *"Days Present"* FROM emp_attendance;
- SELECT c1, c1 / 1, SUBSTR(c1, LENGTH(c1 / 1) + 1) FROM numberString;
- SELECT c1, REVERSE(c1) / 1, LENGTH(REVERSE(c1) / 1), REVERSE(SUBSTR(REVERSE(c1), LENGTH(REVERSE(c1) / 1) + 1)) FROM Stringnumber;
- UPDATE emp SET job := REPLACE(job, 'officers', 'Officers');

mathematical functions

mathematical functions

Syntax	Result
ABS(x)	Returns the absolute value of X.
CEIL(x)	CEIL() is a synonym for CEILING().
CEILING(x)	Returns CEIL value.
FLOOR(x)	Returns FLOOR value.
MOD(n, m), n % m, n MOD m	Returns the remainder of N divided by M. MOD(N,0) returns NULL.
POWER(x, y)	This is a synonym for POW().
RAND()	Returns a random floating-point value
ROUND(x) ROUND(x, d)	Rounds the argument X to D decimal places. The rounding algorithm depends on the data type of X. D defaults to 0 if not specified. D can be negative to cause D digits left of the decimal point of the value X to become zero.
TRUNCATE(x, d)	Returns the number X, truncated to D decimal places. If D is 0, the result has no decimal point or fractional part. D can be negative to cause D digits left of the decimal point of the value X to become zero.

mathematical functions

e.g.

- SELECT CEIL(1.23);
- SELECT CEIL(-1.23);
- SELECT FLOOR(1.23);
- SELECT FLOOR(-1.23);
- SELECT ROUND(-1.23);
- SELECT ROUND(-1.58);
- SELECT FLOOR(1 + RAND() * 100); // between 1 to 100
- SELECT FLOOR(20 + RAND() * 56); // between 20 to 75
- SELECT FLOOR(899999 + RAND() * 100000) *OTP*; // 6 digit
- SELECT weight, TRUNCATE(weight, 0) AS *kg*, MID(weight, INSTR(weight, ".") + 1) AS *gms* FROM mass_table;
- SELECT weight, TRUNCATE(weight, 0) AS *kg*, RIGHT(MOD(weight , 1), 2) AS *gms* FROM mass_table;

random characters / number

FLOOR(*start_value* + RAND() * difference between (*start_value* – *end_value*) + 1);

- CAST(CHAR(FLOOR(65 + RAND() * 26)) AS CHAR) → Random uppercase letter (A–Z)
- CAST(CHAR(FLOOR(97 + RAND() * 26)) AS CHAR) → Random lowercase letter (a–z)
- FLOOR(0 + RAND() * 10)) → Random digit (0–9)

random numbers between

- 1 to 100 → FLOOR(1 + RAND() * 100);
- 1 to 14 → FLOOR(1 + RAND() * 14);
- 20 to 75 → FLOOR(20 + RAND() * 56);
- 6 digit OPT number → FLOOR(899999 + RAND() * 100000);